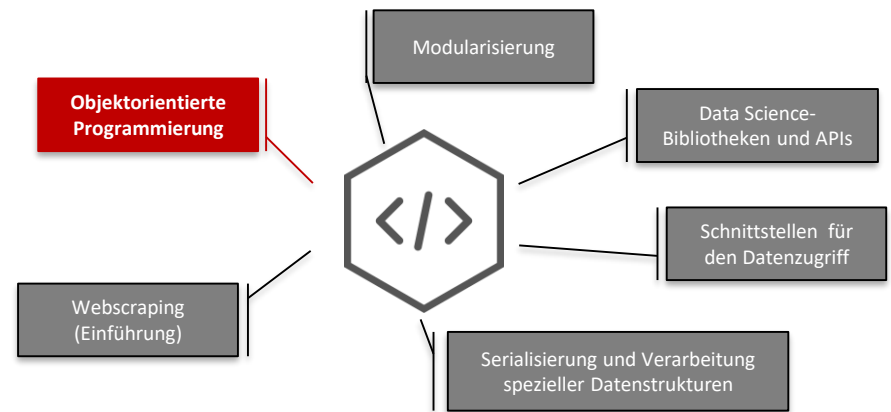


Prof. Dr. Peer Küppers

DSCB230
Informatik für Data Science II

VL2 – Grundlagen der Objektorientierten
Programmierung (OOP)

Objektorientierte Programmierung (OOP)



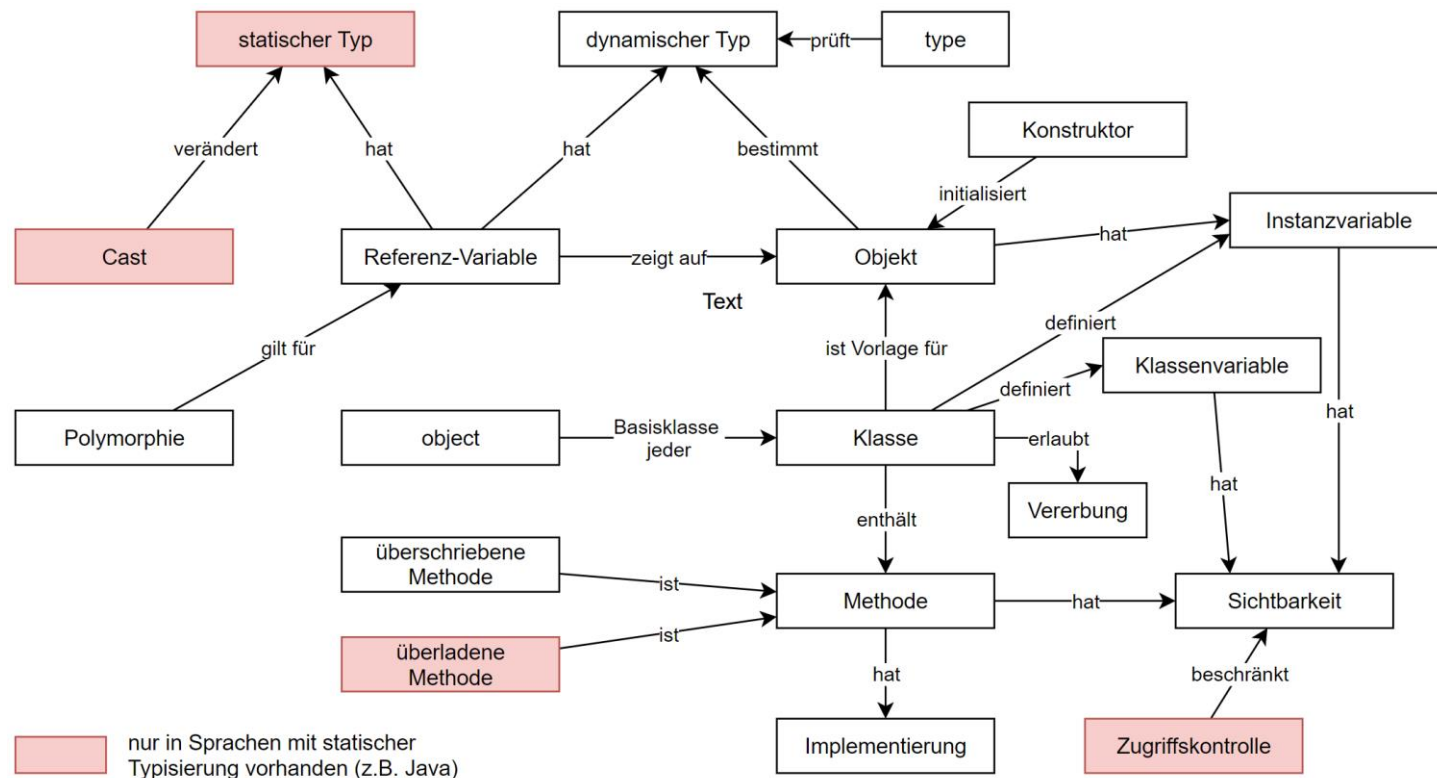
- Die grundlegenden Ideen und Konzepte der OOP verstehen sowie in Python umsetzen können.
 - Klassen definieren sowie Objekte erzeugen können. Spezielle Methoden nutzen, um das Objekt-Handling zu erleichtern.
 - Den Sinn der Vererbung durchdringen und Klassenhierarchien implementieren können.
 - Weitere Ansätze aus der OOP kennenlernen (abstrakte Klassen und Schnittstellen).
- Das Konzept von Exceptions verstehen und damit das Fehlerhandling in Ihrer Software optimieren.
- Die Zusammenhänge zwischen Datenstrukturen sowie Analytics und der OOP verinnerlichen.
 - In diesem Kontext das Konzept von Iteratoren umsetzen lernen.

Agenda

- **Grundlagen und Terminologie**
- ☐ **Klassen und Objekte**
- ☐ **Vererbung und Polymorphie**
- ☐ **Abstrakte Klassen, Schnittstellen, Exceptions**
- ☐ **Datenstrukturen und OOP, Iteratoren**
- ☐ **Zusammenfassung & Ausblick**

Terminologie – Grundbegriffe

- Grundlegende Zusammenhänge, die wir kennenlernen werden



Terminologie – Grundbegriffe

- Ziele der Objektorientierten Programmierung (OOP)
 - Komplexität in (großen) Programm-Codes reduzieren
 - Fehleranfälligkeit verringern
 - Wiederverwendbarkeit verbessern (=Programmieraufwand reduzieren)

- Grundideen
 - **Zusammengehörige Daten und Funktionen** zu möglichst übersichtlichen, wiederverwendbaren Einheiten gruppieren (Objekte)
 - Objekt: ein Element, das Funktionen, Prozeduren und einen inneren Zustand besitzt
 - Klasse: Vorlagen für gleichartige Objekte („Konstruktionsplan“ für Objekte)
 - Verwendung dieser Objekte nur über eine **öffentliche Schnittstelle**, ohne dass Aufrufer über Implementierungsdetails Bescheid wissen müssen
 - **Keine direkte Manipulation des internen Zustands** (=der internen Daten eines Objekts) von Objekten „von außen“; nur die öffentliche Schnittstelle nutzen

Vgl. [Prinzipien des objektorientierten Designs](#)

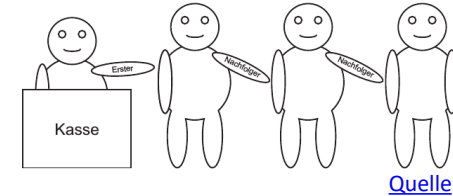
Terminologie – Grundbegriffe

- Grundlegende Konzepte der Objektorientierten Programmierung
 - **Kapselung**
 - Der Zustand von Objekten ist privat und wird nicht von anderen Objekten manipuliert.
 - Nur veröffentlichte Methoden von Objekten können von anderen Objekten aufgerufen werden.
 - **Abstraktion**
 - Implementierungsdetails bleiben für aufrufende Objekte verborgen – nur relevante Operationen werden mit einer entsprechenden Schnittstelle zur Verfügung gestellt
 - **Vererbung**
 - Übertragung von Eigenschaften einer Klasse auf andere Klassen
 - **Polymorphismus**
 - Variablen können sich vielgestaltig (polymorph) verhalten, d.h. sie kann auf Objekte verschiedenen Typs zeigen
- Diese Aspekte werden wir in diesem Fach kennenlernen.

Übung

■ Programmieren Sie in einer Python-IDE (bspw. VSCode) eine „Warteschlange“.

- Dies soll die Warteschlange an einer Kasse simulieren.
- Es sollen Kunden „ankommen“ können und sich so in die Warteschlange einreihen.
- Es sollen Kunden die Warteschlange „verlassen“ können und aus dieser somit „entfernt“ werden.
- Die Warteschlange soll ausgegeben werden können.



■ Dies könnte eine Lösung darstellen:

```
warteschlange = []
```

```
ankommen(warteschlange, "Peter")
ankommen(warteschlange, "Julia")
ankommen(warteschlange, "Manfred")
```

```
ausgabe(warteschlange)
verlassen(warteschlange)
ausgabe(warteschlange)
```

```
def ankommen(w, neuer_kunde):
    """ Hängt einen Kunden an die Warteschlange w (Typ list) an. """
    w.append(neuer_kunde)
```

```
def verlassen(w):
    """ der erste Kunde in der Warteschlange wird "entfernt".
        Gibt den Kunden als Rückgabewert zurück """
    if len(w) > 0:
        kunde = w.pop() # erster in der Warteschlange
        print(f"{kunde} hat die Warteschlange verlassen")
        return kunde
    else:
        print(f"Kein Kunde in der Warteschlange")
```

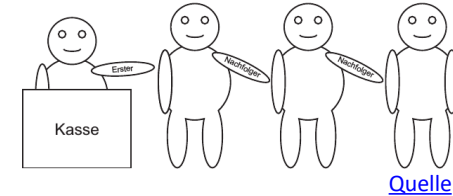
■ Diskutieren Sie die Lösung.

```
def ausgabe(w):
    """ zeigt die Warteschlange an """
    print("Warteschlange:")
    for idx, kunde in enumerate(w):
        print(f"Position {idx}: {kunde}")
```

Übung

- Legen Sie in einer Python-IDE (bspw. VSCode) eine Klasse „Warteschlange“ an.

- Die Klasse soll die Warteschlange an einer Kasse simulieren.
- Es sollen Kunden „ankommen“ können und sich so in die Warteschlange einreihen.
- Es sollen Kunden die Warteschlange „verlassen“ können und aus dieser somit „entfernt“ werden.
- Die Warteschlange soll ausgegeben werden können.



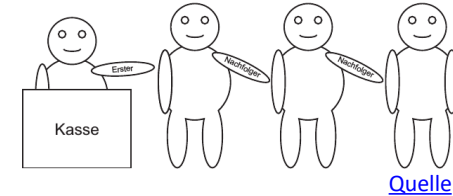
- Anstelle einer manuellen Implementierung mit Listen möchten wir höhere Wiederverwendbarkeit, eine saubere Trennung des internen Warteschlangen-Zustands und damit Kapselung erreichen. D.h. die Nutzung der Warteschlange sollte eher so aussehen:

```
w = Warteschlange() # Objekt von Klasse "Warteschlange" instanziiieren
# einige Kunden "kommen an"
w.ankommen("Peter")
w.ankommen("Julia")
w.ankommen("Manfred")

w.ausgabe() # Warteschlange anzeigen
w.verlassen() # der nächste Kunde verlässt die Warteschlange
w.ausgabe() # Warteschlange anzeigen
```


Übung

- Legen Sie in einer Python-IDE (bspw. VSCode) eine Klasse „Warteschlange“ an.



```
class Warteschlange():
    """ Die Klasse bildet eine Warteschlange ähnlich zu der eines Supermarkts ab """

    def __init__(self):
        """ Konstruktor: legt eine leere Warteschlange an """
        self.kunden = []

    def ankommen(self, kunde):
        """ Einen Kunden an die Warteschlange anfügen """
        self.kunden.append(kunde)

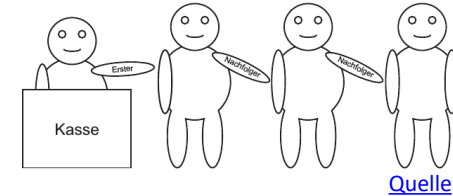
    def verlassen(self):
        """ Der nächste Kunde in der Warteschlange verlässt diese und wird von der Methode zurückgegeben """
        if len(self.kunden) > 0:
            kunde = self.kunden.pop(0)
            print(f"{kunde} hat die Warteschlange verlassen")
            return kunde
        else:
            print(f"Kein Kunde in der Warteschlange")
            return None

    def ausgabe(self):
        """ Die Warteschlange darstellen """
        print("Warteschlange:")
        for idx, kunde in enumerate(self.kunden):
            print(f"Position {idx}: {kunde}")
```

Übung

■ Legen Sie in einer Python-IDE (bspw. VSCode) eine Klasse „Kunde“ an.

- Anstelle der Repräsentation von Kunden über einen einfachen String möchten wir nun auch Kunden über Objekte repräsentieren.
 - Kunden sollen einen Vor- und Nachnamen haben.
 - Wenn sie auf der Konsole ausgegeben werden (print), dann sollen diese beiden Attribute angezeigt werden.



```
class Kunde():  
    """ Die Klasse repräsentiert Kunden """  
  
    def __init__(self, vorname, nachname):  
        self.vorname = vorname  
        self.nachname = nachname  
  
    def __str__(self):  
        """ Ausgabe eines Kunden durch Vor- und Nachnamen """  
        return f"{self.vorname} {self.nachname}"
```

- Nutzen Sie denselben Code wie oben, aber fügen Sie in die Warteschlange Kunde-Objekte ein.

```
w2 = Warteschlange()  
w2.ankommen(Kunde("Peter", "Müller"))  
w2.ankommen(Kunde("Julia", "Schulze"))  
w2.ankommen(Kunde("Manfred", "Mommsen"))  
w2.ausgabe()
```

Terminologie – Grundbegriffe

■ Zusammenhang zum Beispiel

■ Kapselung

- Der Zustand von Objekten ist privat und wird nicht von anderen Objekten manipuliert.
 - **Die Warteschlange wird nicht von „außen“ als Parameter (Liste) übergeben. Die Datenstruktur ist Objekt-intern und könnte völlig anders umgesetzt werden.**
- Nur veröffentlichte Methoden von Objekten können von anderen Objekten aufgerufen werden.
 - **Dies ist in Python nicht so strikt umsetzbar wie in bspw. Java. Solange man sich an die Konventionen hält funktioniert aber alles genauso.**

■ Abstraktion

- Implementierungsdetails bleiben für aufrufende Objekte verborgen – nur relevante Operationen werden mit einer entsprechenden Schnittstelle zur Verfügung gestellt
 - **S.o.: wir könnten die interne Datenstruktur der Warteschlange ändern und die öffentliche Schnittstelle (API) gleich gestalten.**

■ Vererbung

- Übertragung von Eigenschaften einer Klasse auf andere Klassen
 - **Dies werden wir noch kennenlernen.**

■ Polymorphismus

- Variablen können sich vielgestaltig (polymorph) verhalten, d.h. sie kann auf Objekte verschiedenen Typs zeigen
 - **Dies werden wir noch kennenlernen.**

Agenda

- ☐ Grundlagen und Terminologie
- ☒ **Klassen und Objekte**
- ☐ Vererbung und Polymorphie
- ☐ Abstrakte Klassen, Schnittstellen, Exceptions
- ☐ Datenstrukturen und OOP, Iteratoren
- ☐ Zusammenfassung & Ausblick

Klassen und Objekte

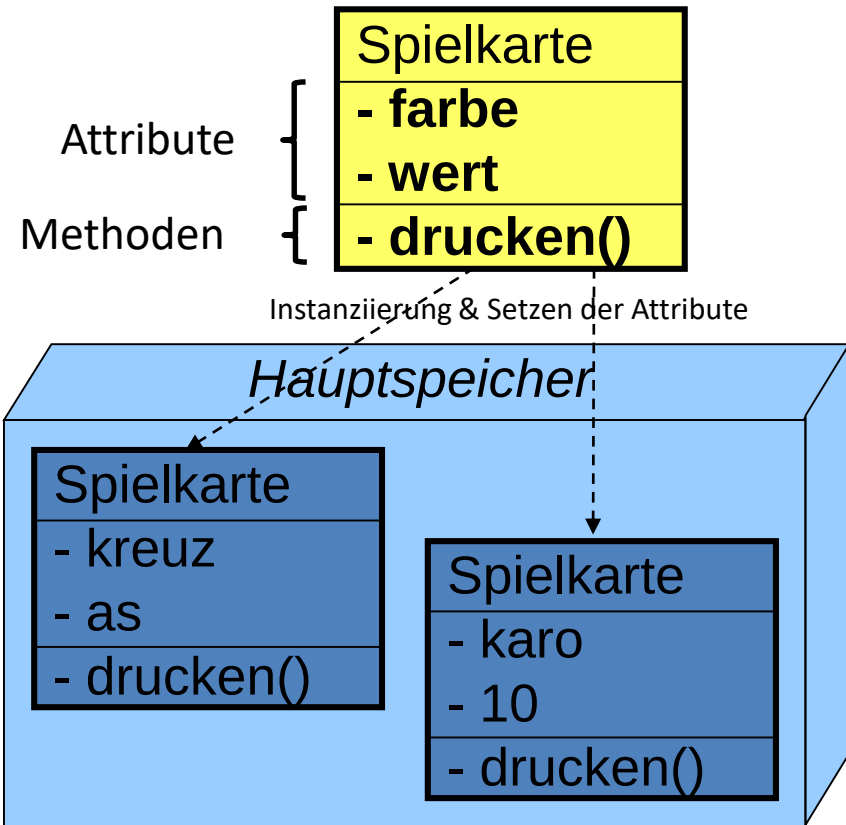
Klasse

- Vorlage (Schablone, Typ) für Objekte
- Definiert
 - Attribute = Charakteristika
 - Methoden = Fähigkeiten, Funktionen

Objekte

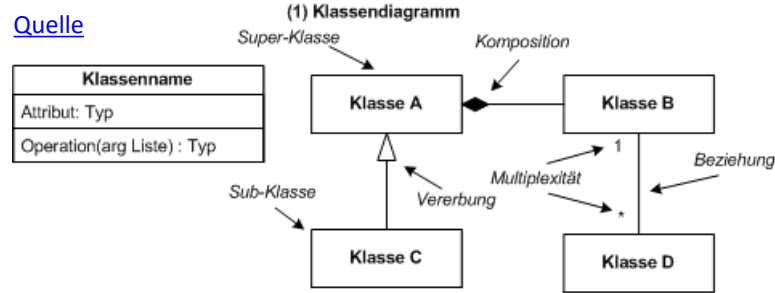
- Eine aus der Klasse abgeleitete (instanzierte) Einheit im Hauptspeicher
- Mehrere Objekte einer Klasse können instanziiert werden.
 - Diese haben dieselbe Struktur, können jedoch unterschiedliche Zustände aufweisen
 - Zustand = „Konkrete Werte der Attribute“

Klasse in UML-Notation:
(vgl. nächste Folie)

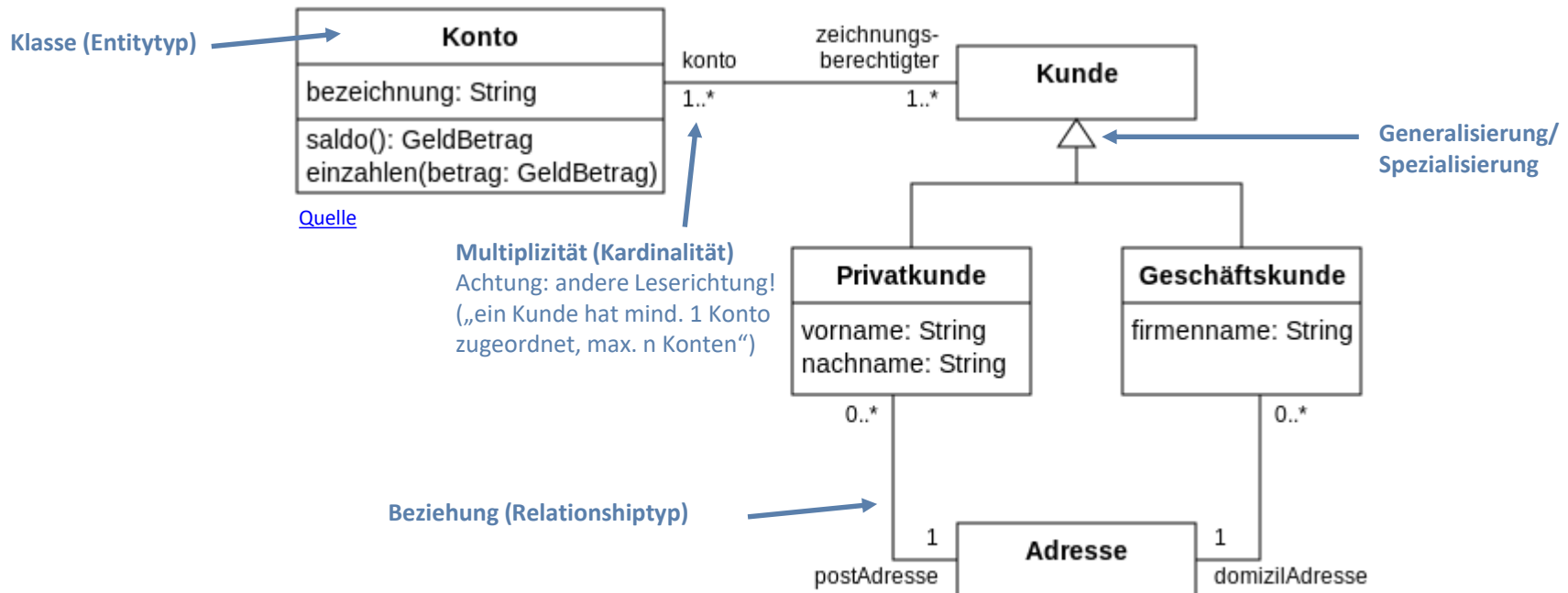


Begleitmaterialien

Quelle



- Zu UML vgl. [Link](#)
 - Ermöglicht die Modellierung von Datenstrukturen analog zu ERM
 - Erweiterung um Datenverarbeitungs-bezogene Konzepte, insb. Methoden



Klassen und Objekte

Umsetzung in Python

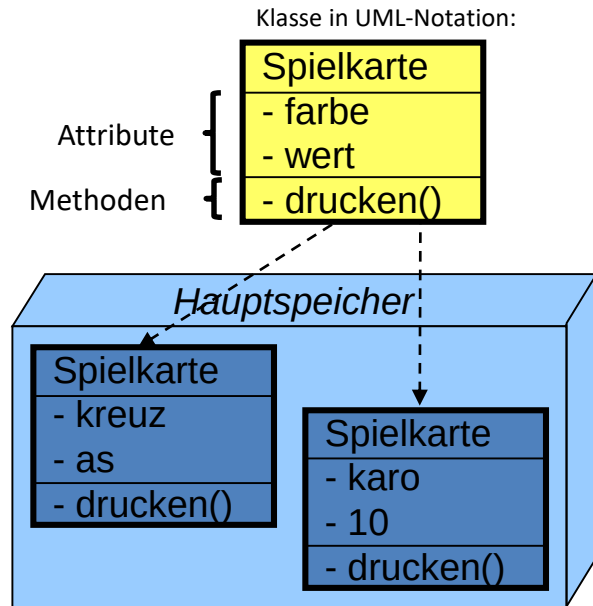
- Einleitung der Klassendefinition mit `class`
- Zugriff auf Attribute mit `self.attributname`
- Methoden erhalten immer den Parameter `self`
 - Dies ist später die Referenz auf das generierte Objekt.
 - In Java wird dies als „this“ bezeichnet.

```
class Spielkarte():  
    """ Klasse zur Repräsentation von Spielkarten """  
  
    def drucken(self):  
        """ Gibt Farbe und Werte auf der Konsole aus """  
        print(f"Spielkarte mit {self.farbe} - {self.wert}")
```

```
k1 = Spielkarte()  
k1.farbe = "kreuz"  
k1.wert = "as"
```

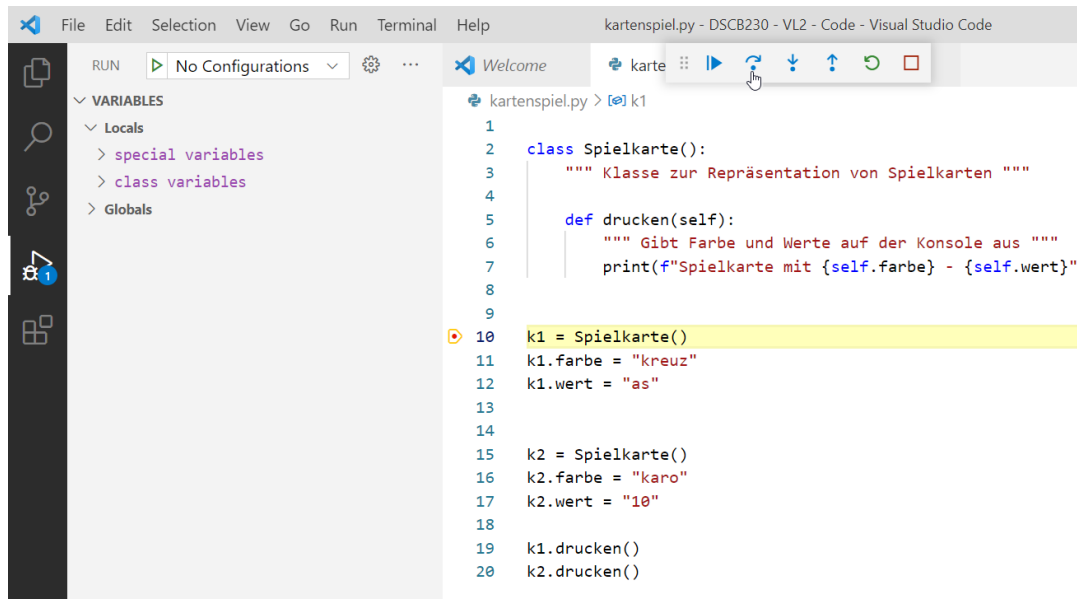
```
k2 = Spielkarte()  
k2.farbe = "karo"  
k2.wert = "10"
```

```
k1.drucken()  
k2.drucken()
```



Übung

- Legen Sie die Klasse Spielkarte in einer Datei „kartenspiel.py“ an.
 - Starten Sie Ihr Programm und betrachten Sie die Ausgabe (in VSCode Strg + F5).
 - Setzen Sie einen „Breakpoint“ auf Zeile 10, d.h. die Instanziierung der ersten Spielkarte.
 - Links neben die „10“ klicken, so dass ein roter Punkt dort erscheint.
 - Starten Sie nun Ihr Programm im Debug-Modus (in VSCode F5).
 - Drücken Sie Enter (Auswahl von „Python File“)

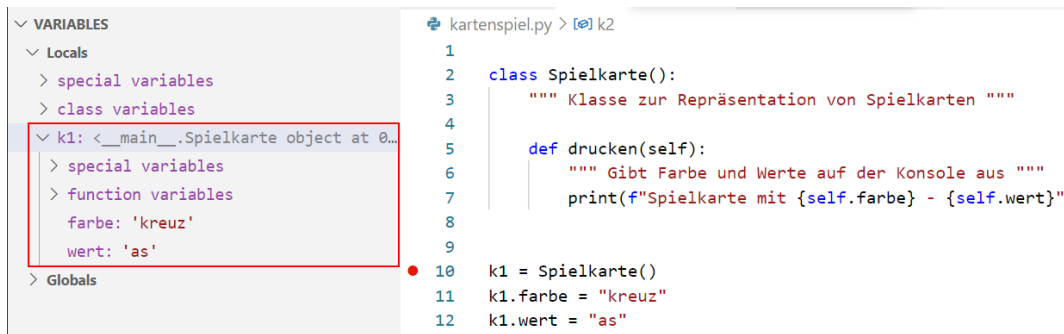


```
1 class Spielkarte():
2     """ Klasse zur Repräsentation von Spielkarten """
3
4     def drucken(self):
5         """ Gibt Farbe und Werte auf der Konsole aus """
6         print(f"Spielkarte mit {self.farbe} - {self.wert}")
7
8
9
10 k1 = Spielkarte()
11 k1.farbe = "kreuz"
12 k1.wert = "as"
13
14
15 k2 = Spielkarte()
16 k2.farbe = "karo"
17 k2.wert = "10"
18
19 k1.drucken()
20 k2.drucken()
```


Übung

■ Arbeiten Sie weiter in „kartenspiel.py“

- Führen Sie Zeilen 10-12 aus (Step Over, F10) und betrachten Sie die Objekte und Attribute.



```
▼ VARIABLES
  ▼ Locals
    > special variables
    > class variables
    ▼ k1: <__main__.Spielkarte object at 0...
      > special variables
      > function variables
      farbe: 'kreuz'
      wert: 'as'
    > Globals

kartenspiel.py > [E] k2
1
2 class Spielkarte():
3     """ Klasse zur Repräsentation von Spielkarten """
4
5     def drucken(self):
6         """ Gibt Farbe und Werte auf der Konsole aus """
7         print(f"Spielkarte mit {self.farbe} - {self.wert}")
8
9
10 k1 = Spielkarte()
11 k1.farbe = "kreuz"
12 k1.wert = "as"
```

- Inwiefern wurde in der bisherigen Implementierung die Kapselung (Trennung „innerer Objektzustand“ von äußeren Zugriffen) umgesetzt?
- Wie sollte man dies eher umsetzen?
- Führen Sie die weiteren Zeilen aus und betrachten Sie die Objekte.
- Fügen Sie zwei Zeilen ein: „k3 = k2“ und „k3.drucken()“
 - Setzen Sie den Breakpoint auf „k3.drucken()“ und starten Sie den Debugger.
 - Was bewirkt die Variablenzuweisung?
 - Wie viele Objekte haben wir im Hauptspeicher (also im Zugriff unseres Programms)?
 - Manipulieren Sie den Wert von k3 und betrachten Sie die Objekte im Hauptspeicher (siehe nächste Folie).
 - Dies geht direkt unter „VARIABLES“.
 - Achtung: unter VARIABLES erfolgt u.U. keine direkt Aktualisierung der Werte
 - Nutzen Sie die DEBUG-Console, um die Werte von k2 und k3 zu prüfen.

Übung

- Arbeiten Sie weiter in „kartenspiel.py“
 - Analyse der Zustandsmanipulation

Bei Bedarf: Add to Watch, um die Variable zu „beobachten“

DEBUG CONSOLE

Filter (e.g. text, lexclude)

Python: Aktuelle Datei

```
→ k2
<__main__.Spielkarte object at 0x000025143BFB490>
> special variables
> function variables
  farbe: 'karo'
  wert: 'bube'
→ k2.wert
'bube'
```

Konstrukturen

Problem

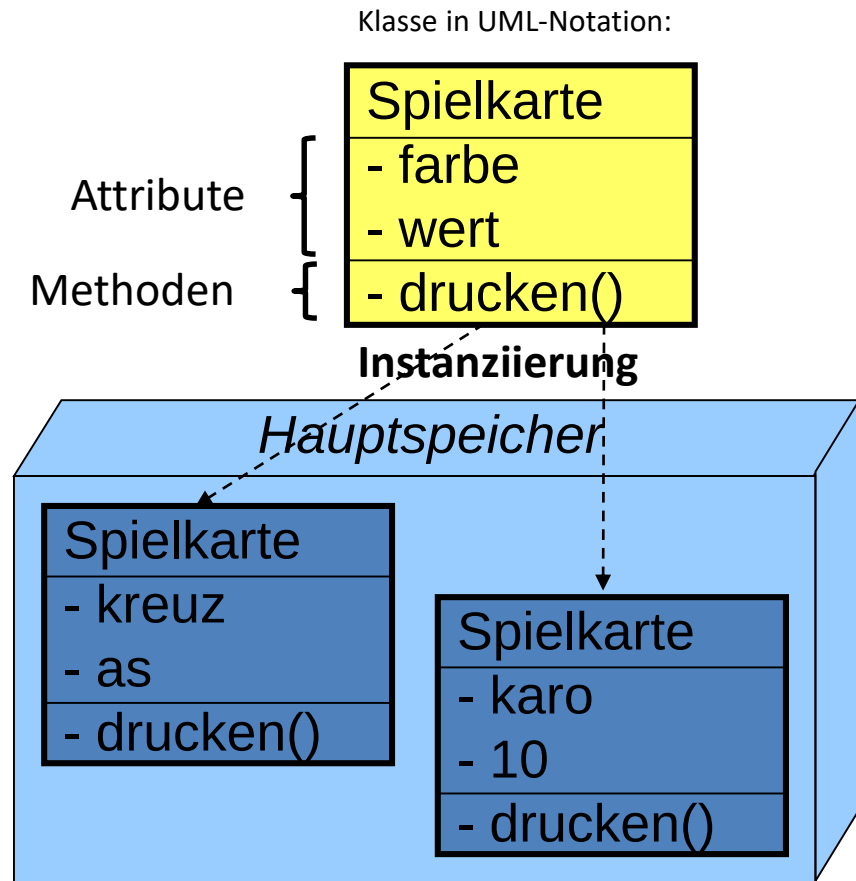
- Wenn wir eine neue Spielkarte instanziiieren, dann sind die Attribute *farbe* und *wert* weder
 - mit sinnvollen Werten versehen, noch
 - deklariert (also als Variablen angelegt).
 - Hinweis: letzteres wäre bspw. in Java kein Problem

Ziel

- Objekte sollen sofort nach ihrer Erzeugung einen sinnvollen Zustand haben.

Lösung

- Konstrukturen = Methoden mit Initialisierungscode



Konstrukturen

Konstrukturen in Python

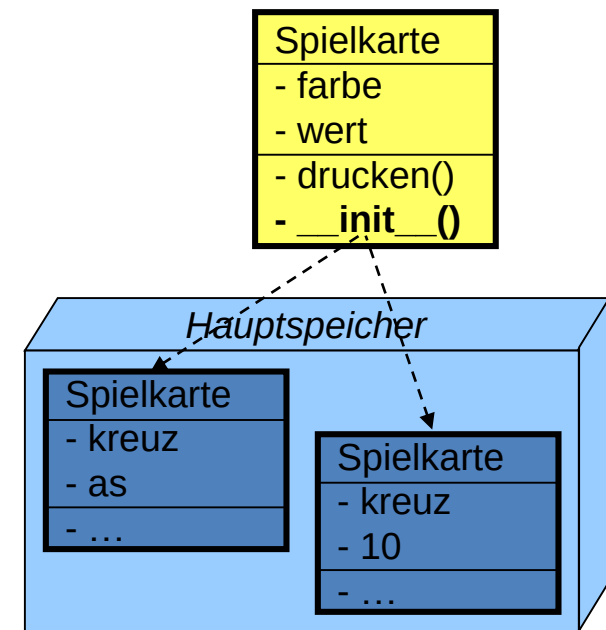
- Werden durch die spezielle Methode `__init__` definiert.

```
class Spielkarte():  
    """ Klasse zur Repräsentation von Spielkarten """  
  
    def __init__(self):  
        """ Konstruktor für eine "Standardkarte" (kreuz as) """  
        self.farbe = "kreuz"  
        self.wert = "as"
```

- Eine neue Instanz von Spielkarte wird also ein „kreuz as“.
- Alternative: übergeben der Werte für „farbe“ und „wert“ bei Instanziierung:

```
def __init__(self, farbe, wert):  
    """ Konstruktor für eine Karte mit übergebenen Argumenten  
        für farbe und wert """  
    self.farbe = farbe  
    self.wert = wert
```

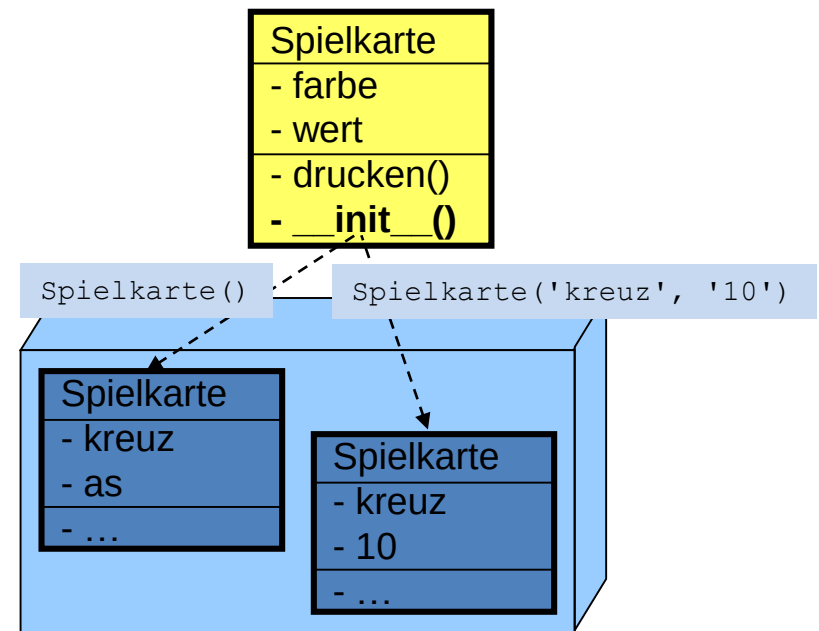
- Ideal wäre die Definition beider Konstrukturen (flexibel nutzbar)
 - **Problem:** Python erlaubt kein Überladen von Methoden (also auch Konstrukturen)
 - Überladen: mehrere Methoden haben denselben Namen, aber verschiedene Parameter
 - Welche Methode aufgerufen wird, wird dann jedem Aufruf anhand der Parameter und ihrer Datentypen automatisch vom Compiler bestimmt.
 - Dies funktioniert in Python nicht!



Konstrukturen

Konstrukturen in Python (cont'd)

- Ideal wäre die Definition beider Konstrukturen (flexibel nutzbar)
 - **Problem:** Python erlaubt kein Überladen von Methoden (also auch Konstrukturen)
 - Überladen: mehrere Methoden haben denselben Namen, aber verschiedene Parameter
 - Welche Methode aufgerufen wird, wird dann jedem Aufruf anhand der Parameter und ihrer Datentypen automatisch vom Compiler bestimmt. Dies funktioniert in Python nicht!
- Einfache Lösung: named arguments (Argumente mit Standardwerten)
 - Python unterscheidet
 - positional arguments (Zuordnung übergebener Werte über die Position in der Argumentliste)
 - named arguments (Übergabe von Werten inklusive Bezeichnung des Parameters)
 - Siehe nächste Folie



Exkurs named vs. positional arguments

Konstrukturen in Python (cont'd)

- Exkurs *positional vs. named arguments*
- Nur *positional arguments*

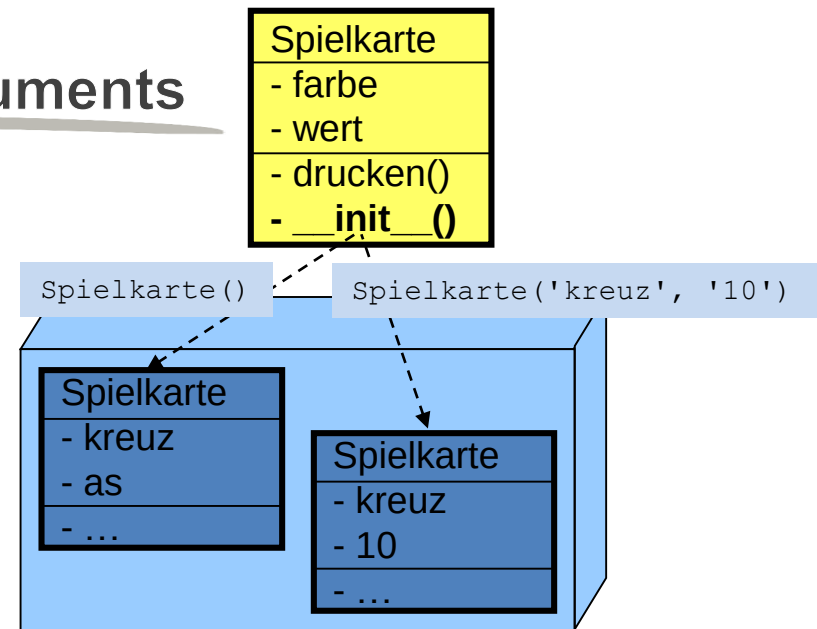
```
def aendern(self, farbe, wert):  
    """ Ändere die Kartenattribute  
        positional --> beide Parameter sind zu übergeben """  
    self.farbe = farbe  
    self.wert = wert
```

- Nur *named arguments* (außer *self*) mit Standardwert *None*

```
def aendern(self, farbe=None, wert=None):  
    """ Ändere die Kartenattribute  
        named --> beide Parameter können übergeben werden (sonst Standard) """  
    self.farbe = farbe  
    self.wert = wert
```

- Mischung (wichtig: erst *positional*, dann *named arguments*)

```
def aendern(self, farbe, wert=None):  
    """ Ändere die Kartenattribute  
        farbe positional --> muss übergeben werden  
        wert named --> kann übergeben werden (sonst Standard) """  
    self.farbe = farbe  
    self.wert = wert
```



Konstruktoren

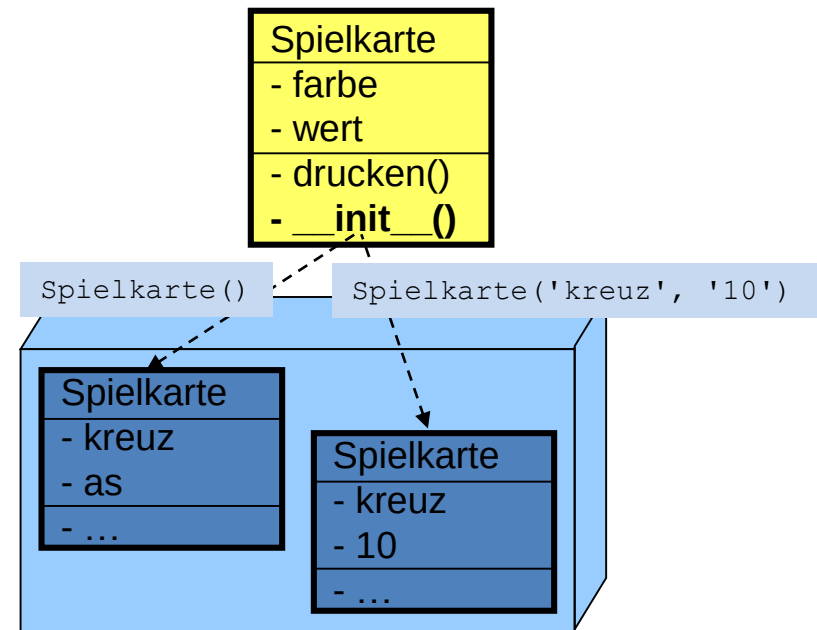
Konstruktoren in Python (cont'd)

- Nutzung von *named arguments* im Konstruktor
- Um unsere Anforderungen zu erfüllen können wir auf named arguments mit Standardwerten in `__init__` zurückgreifen.

```
def __init__(self, farbe='kreuz', wert='as'):  
    """ Konstruktor für eine Karte mit übergebenen Argumenten  
        für farbe und wert. Werden keine Daten übergeben, dann  
        wird ein "kreuz as" angelegt """  
    self.farbe = farbe  
    self.wert = wert
```

Dies ermöglicht:

```
k4 = Spielkarte()  
k5 = Spielkarte('pik', '7')  
k4.drucken()  
k5.drucken()
```



Konstrukturen

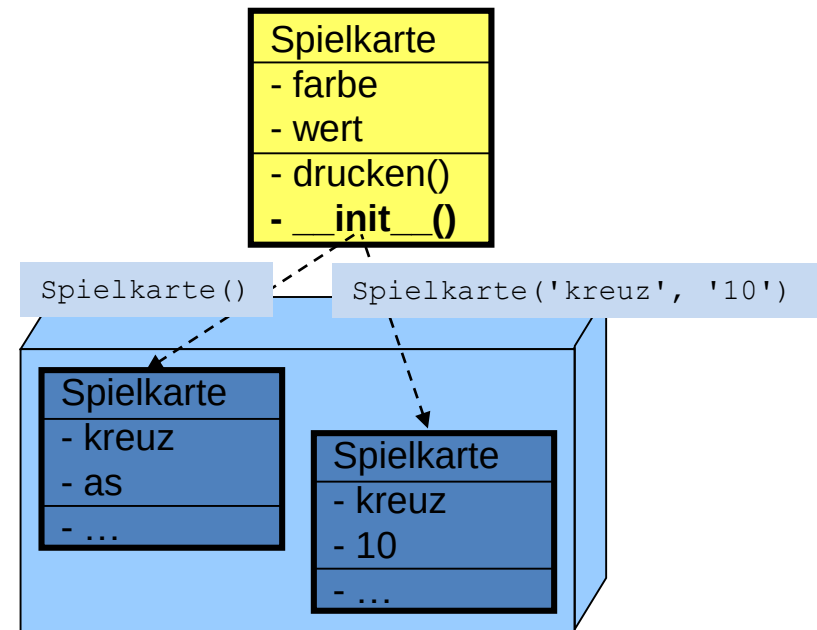
Konstrukturen in Python (cont'd)

- Nutzung von *named arguments* im Konstruktor
- Um unsere Anforderungen zu erfüllen können wir auf named arguments mit Standardwerten in `__init__` zurückgreifen.

```
def __init__(self, farbe='kreuz', wert='as'):  
    """ Konstruktor für eine Karte mit übergebenen Argumenten  
        für farbe und wert. Werden keine Daten übergeben, dann  
        wird ein "kreuz as" angelegt """  
    self.farbe = farbe  
    self.wert = wert
```

Dies ermöglicht:

```
k4 = Spielkarte()  
k5 = Spielkarte('pik', '7')  
k4.drucken()  
k5.drucken()
```



Spielkarte2
- _farbe
- _wert
- drucken()
- __init__()
- get/set

Get-/Set-Methoden

- Ziel: Zugriffe auf „private“ Attribute von außen vermeiden.
 - Der Zustand soll nur von dem Objekt selbst verwaltet werden.
 - Aber was, wenn wir bspw. das Setzen der Farbe erlauben möchten.
- Schritt 1: Konvention zur „Markierung“ privater Attribute / Methoden
 - In Python gibt es keine Sichtbarkeiten wie in bspw. Java. Wir können die Attribute nicht verbergen.
 - Es gibt aber die Konvention, dass Attribute mit vorangestelltem „_“ als privat zu interpretieren sind.

```
class Spielkarte2():  
    """ Klasse zur Repräsentation von Spielkarten (v2) """  
  
    def __init__(self, farbe='kreuz', wert='as'):  
        """ Konstruktor für eine Karte mit übergebenen Argumenten  
            für farbe und wert. Werden keine Daten übergeben, dann  
            wird ein "kreuz as" angelegt """  
        self._farbe = farbe # Markieren als privates Attribut  
        self._wert = wert # Markieren als privates Attribut
```

- Hinweis: Python bietet auch die Möglichkeit, zwei Underscores vor einem Attribut- oder Methodennamen zu definieren.

- Der direkte Zugriff wird dann schwieriger, aber nicht unmöglich!

Get-/Set-Methoden

Spielkarte2
- _farbe
- wert
- drucken()
- __init__()
- get/set

Get-/Set-Methoden (cont'd)

- Schritt 2: get-/set-Methoden mit *Decorators*
 - *Decorators* werden genutzt, um auf relativ einfachem Weg komplexe Programmiermuster umzusetzen
 - → Lesbarkeit erhöhen, Mächtigkeit von Python steigern, um bspw. Funktionalitäten von typisierten Sprachen zu erreichen.
 - Mittels der `@property` und `@attributname.setter` Methoden können wir die beiden Attribute `farbe` und `wert` leicht manipulieren lassen. Die Manipulation läuft aber über Methoden, so dass wir bspw. Wertebereichsprüfungen umsetzen können.

```
@property
def farbe(self):
    """ get-Methode von Attribut farbe """
    print("getter von farbe aufgerufen")
    return self._farbe
```

```
@property
def wert(self):
    """ get-Methode von Attribut wert """
    print("getter von wert aufgerufen")
    return self._wert
```

```
@farbe.setter
def farbe(self, value):
    print("setter von farbe aufgerufen")
    self._farbe = value
```

```
@wert.setter
def wert(self, value):
    print("setter von wert aufgerufen")
    self._wert = value
```

getter/setter ermöglichen folgenden Code:

```
neuekarte = Spielkarte2()
print(neuekarte.farbe) # Zugriff über get-Methode
neuekarte.farbe = 'karo' # Zugriff über set-Methode
```

➡ Konsolenausgabe

```
getter von farbe aufgerufen
karo
setter von farbe aufgerufen
```

String-Repräsentation von Objekten

Spielkarte2
- _farbe
- wert
- drucken()
- __init__()
- get/set
- __str__()

Motivation

- Die Konsolausgabe von `print(ref_var)` liefert die Adresse des Objekts:

`print(neuekarte)` ➡ `<__main__.Spielkarte2 object at 0x0000023DB6216070>`

- Wir möchten nicht an unterschiedlichen Stellen im Code die Formatierung eines Objekts programmieren.

```
print(f"Spielkarte mit Farbe {neuekarte.farbe} und Werte {neuekarte.wert}")
```

- Die *drucken*-Methode war schon ein guter Ansatz. Aber wir möchten nicht immer auf der Konsole die String-Repräsentation von Objekten ausgeben.
- Python bietet dazu die spezielle Methode `__str__`:

```
def __str__(self):  
    return f"Spielkarte mit Farbe {self.farbe} und Werte {self.wert}"
```

`print(neuekarte)` ➡ `Spielkarte mit Farbe karo und Werte as`

- Wir sollten dementsprechend auch die *drucken*-Methode anpassen

```
def drucken(self):  
    """ gibt den String der Spielkarte auf der Konsole aus """  
    print(self)
```



Hier sehen wir die Verwendung von `self` → `__str__` wird implizit aufgerufen!

Objektvergleich

Spielkarte2
- _farbe
- wert
- drucken()
- __init__()
- get/set
- __str__()
- __eq__()

Motivation

- Es ist üblicherweise in Python die Anforderung, dass der Operator „==“ die Gleichheit von Objekten im Hinblick auf ihren Zustand (also die Werte von Attributen) prüfen soll.

- Beispiel:

```
x1 = [10, 20, 30]
x2 = [10, 20, 30]
print(f"Is x1 == x2? {x1 == x2}") # liefert True!
```

- x1 und x2 sind unterschiedliche Objekte (auch wenn wir das bisher ignoriert haben)
- Der Vergleich mit „==“ liefert True! Wenn wir prüfen wollen, ob zwei Variablen auf das identische Objekt referenzieren, müssen wir „is“ verwenden:

```
x3 = x1
print(f"Result of 'x1 is x2'? {x1 is x2}") # liefert False!
print(f"Result of 'x1 is x3'? {x1 is x3}") # liefert True!
```

Problem

- Dieses Muster ist für unsere Klasse nicht umgesetzt:

```
neuekarte2 = Spielkarte2() # kreuz as
neuekarte3 = Spielkarte2() # kreuz as
print(neuekarte2 == neuekarte3) # liefert False!
```

Objektvergleich

Spielkarte2
- _farbe
- wert
- drucken()
- __init__()
- get/set
- __str__()
- __eq__()

Lösung

- Da die Objekte „nicht wissen“ können, wie sie sich untereinander vergleichen sollen, müssen wir dies explizit programmieren!
- Python nutzt hierfür die spezielle Methode `__eq__`
 - → Implementierung der Prüfung auf logische Gleichheit

```
def __eq__(self, other):  
    """ logische Prüfung auf Gleichheit von zwei Spielkarten """  
    return self.farbe == other.farbe and self.wert == other.wert
```

- Hinweise:
 - Sobald diese Vergleichsmethode implementiert ist, funktioniert auch `!=`
 - Der „is“-Vergleich funktioniert weiterhin wie gehabt (Prüfung auf Objektidentität)

```
print(neuekarte2 == neuekarte3) # liefert True nach Programmierung von __eq__  
print(neuekarte2 != neuekarte3) # liefert False nach Programmierung von __eq__  
print(neuekarte2 is neuekarte3) # liefert False!
```

Statische Methoden

Spielkarte2
- __farbe
- __wert
- drucken()
- __init__()
- get/set
- __str__()
- __eq__()
- zufallskarte()

- Bisher: alle Fähigkeiten (Methoden) sind an ein konkretes Objekt gebunden.
 - Der Aufruf einer Methode bedingte somit die Erzeugung eines Objekts.
- Es gibt aber Anforderungen, bei denen Methoden der Klasse ohne ein vorher zu erzeugendes Objekt aufgerufen werden sollen.
 - Beispielzwecke: Hilfsmethoden (bspw. Umrechnungen), Erzeugung von Objekten inkl. Sicherstellung korrekter/gewünschter Werte, Umsetzung bestimmter Software-Entwurfsmuster (bspw. Factory oder Singleton).
 - Unser Beispiel: wir möchten eine zufällige und korrekte Karten für unser Kartenspiel erzeugen und als Objekt zurückgeben lassen (Achtung: „import random“ ist nötig):

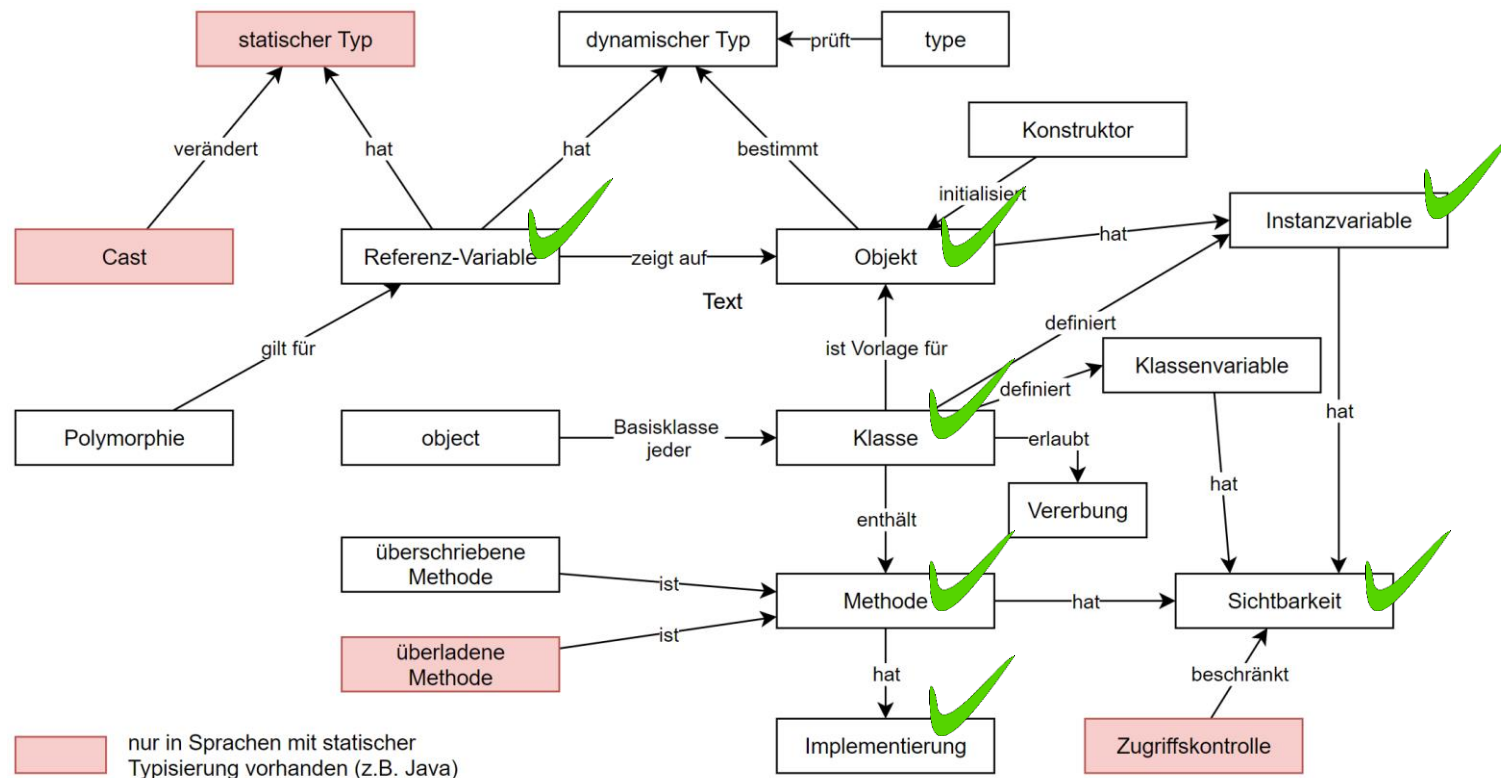
```
@staticmethod
def zufallskarte():
    """ erzeugt eine zufällige Spielkarte """
    farben = ['kreuz', 'pik', 'karo', 'herz']
    zufallsfarbe = random.choice(farben) # ziehe zufällige Farbe
    werte = [7, 8, 9, 10, 'bube', 'dame', 'koenig', 'as']
    zufallswert = random.choice(werte) # ziehe zufälligen Wert
    return Spielkarte(zufallsfarbe, zufallswert)
```

- Wir können dann diese Methode anhand der Klasse aufrufen:

```
zufallskarte = Spielkarte2.zufallskarte()
zufallskarte.drucken()
```

Terminologie – Grundbegriffe

■ Grundlegende Zusammenhänge

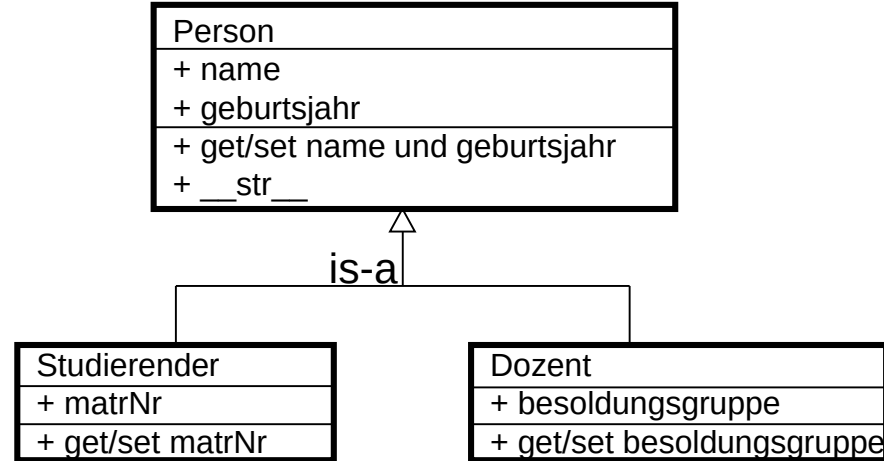


Agenda

- ☐ Grundlagen und Terminologie
- ☐ Klassen und Objekte
- ☒ Vererbung und Polymorphie
- ☐ Abstrakte Klassen, Schnittstellen, Exceptions
- ☐ Datenstrukturen und OOP, Iteratoren
- ☐ Zusammenfassung & Ausblick

Vererbung – Grundlagen

■ Beispiel: Hochschulangehörige



Spezialisierung
↓
↑
Generalisierung

■ Terminologie

- Person wird als **Superklasse** von Studierende und Dozent bezeichnet.
- Studierende und Dozent sind **Subklassen** von Person.

■ Vererbung von Eigenschaften und Methoden

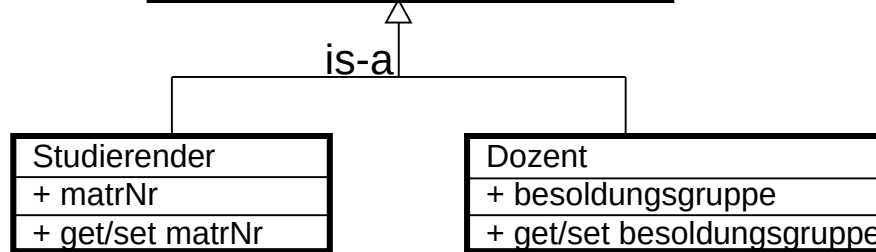
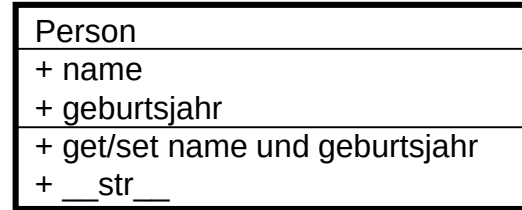
- Jede Eigenschaft der Superklasse gilt automatisch auch für die Subklasse
 - Ein Dozent ist gleichzeitig eine Person und hat auch einen Namen.
- Methoden werden ebenfalls vererbt.
- Die Klassen Studierender und Dozent können beides aber erweitern oder ändern.
 - Die Änderung von Methoden in Subklasse nennt sich „Überschreiben“.

■ Sichtbarkeit geerbter Attribute / Methoden

- Auf private Attribute und Methoden einer Superklasse können Subklassen nicht zugreifen. In Python kann dies aber nicht erzwungen werden!

Vererbung – Grundlagen

- Beispiel:
Hochschulangehörige
- Python-Syntax
 - Die Superklasse, von der geerbt werden soll, wird bei der Klassendefinition angegeben:



Spezialisierung
Generalisierung

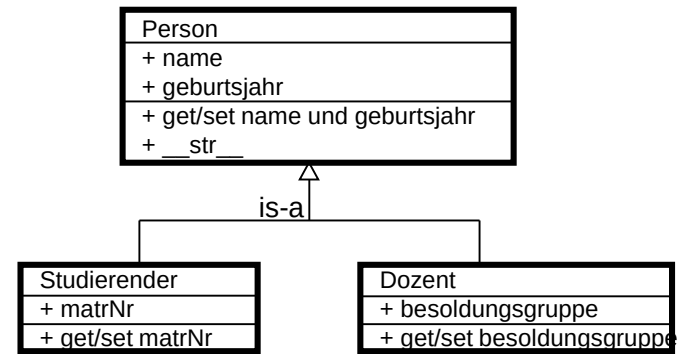
```
class Person():  
    """ Klasse zur Repräsentation von Personen """  
  
    def __init__(self, name=None, geburtsjahr=None):  
        self._name = name  
        self._geburtsjahr = geburtsjahr
```

```
class Studierender(Person):  
    """ Klasse Studierender als Spezialisierung von Person """  
    def __init__(self, name=None, geburtsjahr=None, matrNr=None):  
        super().__init__(name=name, geburtsjahr=geburtsjahr)  
        self._matrNr = matrNr
```

```
class Dozent(Person):  
    """ Klasse Dozent als Spezialisierung von Person """  
    def __init__(self, name=None, geburtsjahr=None, besoldungsgruppe=None):  
        super().__init__(name=name, geburtsjahr=geburtsjahr)  
        self._besoldungsgruppe = besoldungsgruppe
```

Mit super() kann auf Attribute und Methoden der Superklasse zugegriffen werden (hier der Konstruktor)

Vererbung – Grundlagen

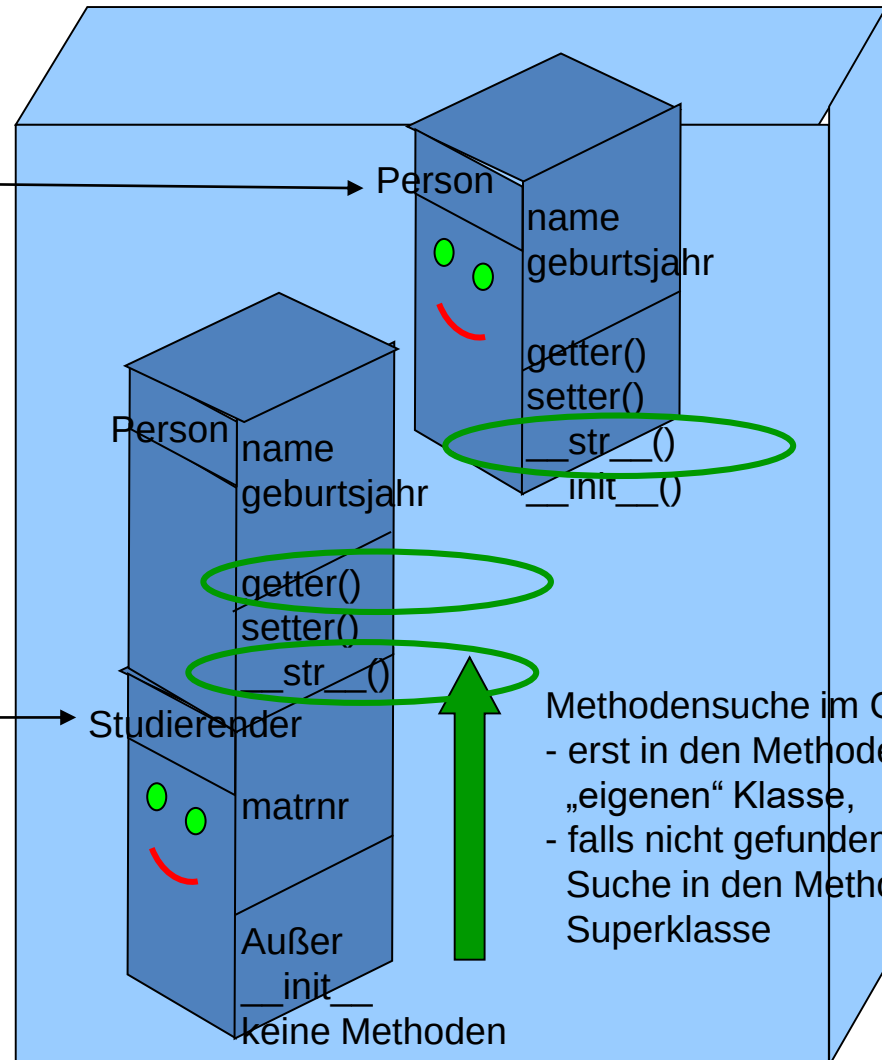


Spezialisierung
Generalisierung

■ Aufrufhierarchie Überschreiben von Methoden

`pers1 = Person(...)`
- `print(pers1)`: nutzt `__str__`

`Stud1 = Student(...)`
- `stud1.geburtsjahr = 1999`: in Klasse `Studierender` wird der `setter` nicht gefunden, Suche in Superklasse `Person`: ok
- `__str__`: ebenso



Methodensuche im Objekt:
- erst in den Methoden der „eigenen“ Klasse,
- falls nicht gefunden, dann Suche in den Methoden der Superklasse

Übung

- Arbeiten Sie in „hochschule.py“
 - Setzen Sie die Klassenhierarchie wie oben dargestellt um.
 - Implementieren Sie für alle Klassen getter/setter-Methoden.
 - Programmieren Sie die `__str__`-Methoden der Subklassen so, dass möglichst viel Code wiederverwendet werden kann.
 - Ausgabe Studierender mit `__str__` soll sein:
Person <name> mit Geburtsjahr <geburtsjahr> und gleichzeitig Studierender mit Matrikelnummer <matrNr>
 - Ausgabe Dozent mit `__str__` soll sein:
Person <name> mit Geburtsjahr <geburtsjahr> und gleichzeitig Dozent mit Besoldungsgruppe <besGr.>
 - Denken Sie dabei an das `super()`-Schlagwort

Übung

■ Lösungsvorschlag

```
class Studierender(Person):
    """ Klasse Studierender als Spezialisierung von Person """
    def __init__(self, name=None, geburtsjahr=None, matrNr=None):
        super().__init__(name=name, geburtsjahr=geburtsjahr)
        self._matrNr = matrNr

    @property
    def matrNr(self):
        return self._matrNr
    @matrNr.setter
    def matrNr(self, value):
        self._matrNr = value

    def __str__(self):
        return f"{super().__str__()} und gleichzeitig Studierender mit Matrikelnummer {self.matrNr}"
```

Die Methode `__str__` wurde in der Subklasse überschrieben!
(Es gibt ja auch in der Superklasse diese Methode.)

```
class Dozent(Person):
    """ Klasse Dozent als Spezialisierung von Person """
    def __init__(self, name=None, geburtsjahr=None, besoldungsgruppe=None):
        super().__init__(name=name, geburtsjahr=geburtsjahr)
        self._besoldungsgruppe = besoldungsgruppe

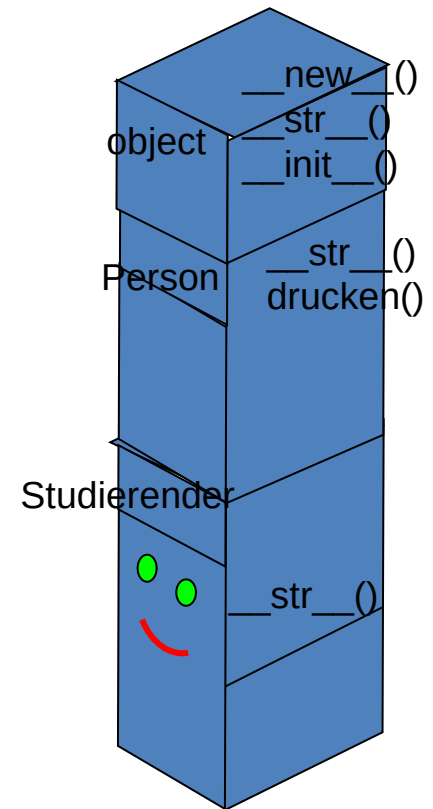
    @property
    def besoldungsgruppe(self):
        return self._besoldungsgruppe
    @besoldungsgruppe.setter
    def besoldungsgruppe(self, value):
        self._besoldungsgruppe = value

    def __str__(self):
        return f"{super().__str__()} und gleichzeitig Dozent mit Besoldungsgruppe {self.besoldungsgruppe}"
```

Die Methode `__str__` wurde in der Subklasse überschrieben!
(Es gibt ja auch in der Superklasse diese Methode.)

Basisklasse object

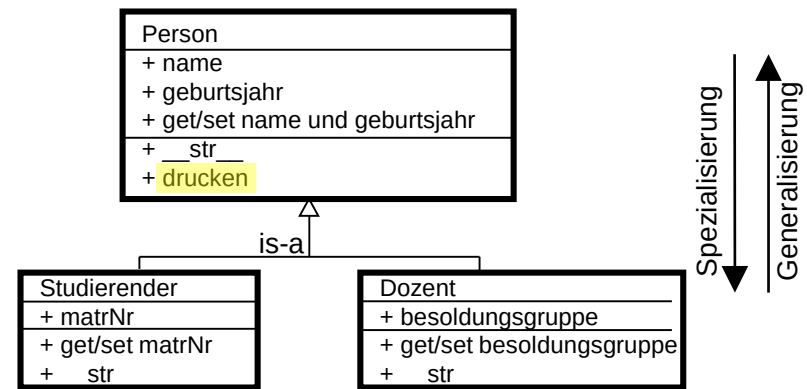
- In Python erben alle Klassen von der Basisklasse *object*
 - Diese bietet allen Klassen die Methoden `__new__`, `__init__`, sowie `__str__`
 - `__new__`: Basismethode zum Instanzieren eines Objekts der Klasse
 - `__init__`: Konstruktor (s.o.)
 - `__str__`: Jedes Objekt kann auf der Konsole als String ausgegeben werden



Polymorphie

- Ein Objekt hat eine einzigartige Form, bestimmt durch die Klasse, die instanziiert wurde.
- Eine Variable kann sich vielgestaltig (polymorph) verhalten: Sie kann auf Objekte verschiedenen Typs zeigen.
- Bei der dynamischen Polymorphie werden in mehreren Klassen einer Vererbungslinie gleiche Methoden unterschiedlich implementiert („überschreiben“).
 - Zur Laufzeit wird entschieden, welche der Methoden ausgeführt wird.
 - Vgl. folgendes Beispiel

Polymorphie



■ Beispiel

- Wir ergänzen unsere Klassenhierarchie durch eine Methode „drucken“ (in Klasse Person zu implementieren), die einfach nur das Objekt auf der Konsole ausgibt.

- Dabei wird also die passende `__str__`-Methode aufgerufen:

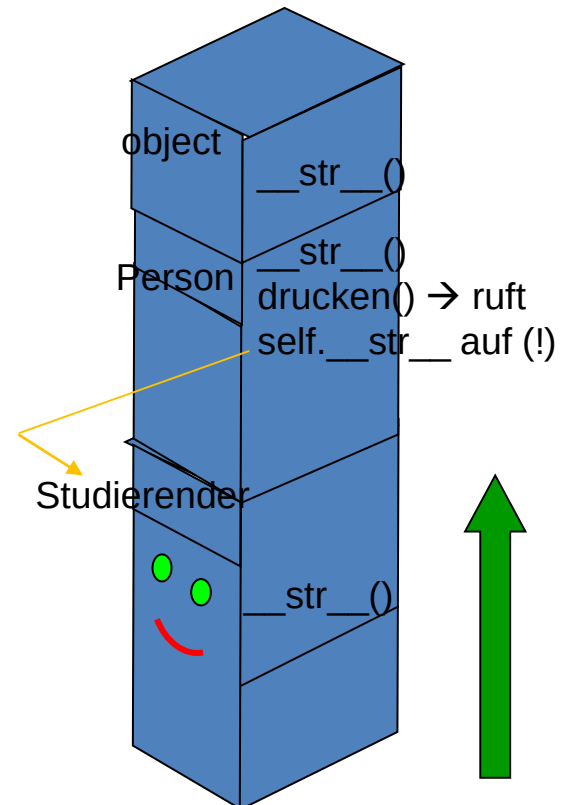
```
def drucken(self):  
    print(self)
```

- Instanzieren Sie einen Studierenden s1 und führen Sie `s1.drucken()` aus.

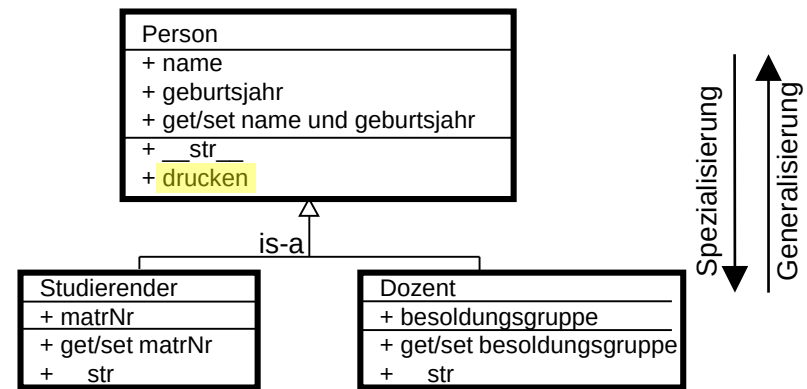
- Was beobachten Sie? Achten Sie genau auf die Ausgabe!

→ `self` in der Methode „drucken“ bezieht sich ursprünglich auf Person.

→ Die Nutzung von `self` im `print`-Statement führt dazu, dass an das Objekt (also ein Studierender) der Aufruf von `__str__` gesendet wird. → es wird die Ausgabe von Studierender gefunden (also `__str__` in Studierender ausgeführt)



Polymorphie



■ Beispiel (cont'd)

- Wir legen nun eine Liste mit drei Personen an:
1 Objekt von Person, 1 Objekt von Studierender, 1 Objekt von Dozent
- Wir iterieren über die Liste, lassen uns den Typ des referenzierten Objekts in der Iteration ausgeben und rufen schließlich „drucken“ auf :

```
s2 = Studierender(name="Petra", geburtsjahr=1999, matrNr=123457)
d1 = Dozent(name="Hans", geburtsjahr=1984, besoldungsgruppe="W2")
p1 = Person(name="Julia", geburtsjahr=2001)
```

```
personen_liste = [s2, p1, d1]
```

```
for p in personen_liste:
    print(type(p))
    p.drucken()
```

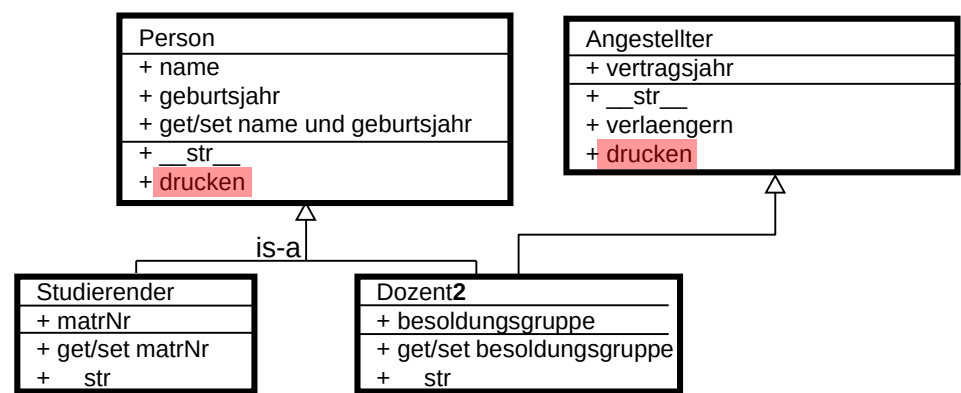
- Die Variable p verhält sich polymorph:

```
Person Petra mit Geburtsjahr 1999 und gleichzeitig Studierender mit Matrikelnummer 123457
<class '__main__.Person'>
Person Julia mit Geburtsjahr 2001
<class '__main__.Dozent'>
Person Hans mit Geburtsjahr 1984 und gleichzeitig Dozent mit Besoldungsgruppe W2
```

Mehrfachvererbung

■ Python erlaubt Mehrfachvererbung

- Die Superklassen werden durch Komma getrennt.
- Bei der „Suche“ nach passenden Methoden in Superklassen wird nach der angegebenen Reihenfolge vorgegangen.
 - So werden Konflikte vermieden, bspw. bzgl. drucken.
- Achtung: anstelle von super() nutzen wir explizite Aufrufe der Superklassen-Methoden (in __init__ und __str__)



```

class Angestellter():
    """ Klasse zur Repräsentation von Angestellten """

    def __init__(self, vertragsjahr=None):
        self._vertragsjahr = vertragsjahr

    def verlaengern(self):
        """ verlängert den Vertrag um ein Jahr """
        self._vertragsjahr += 1

    def __str__(self):
        return f"Angestellter bis {self._vertragsjahr}"

    def drucken(self):
        print(self)
    
```

```

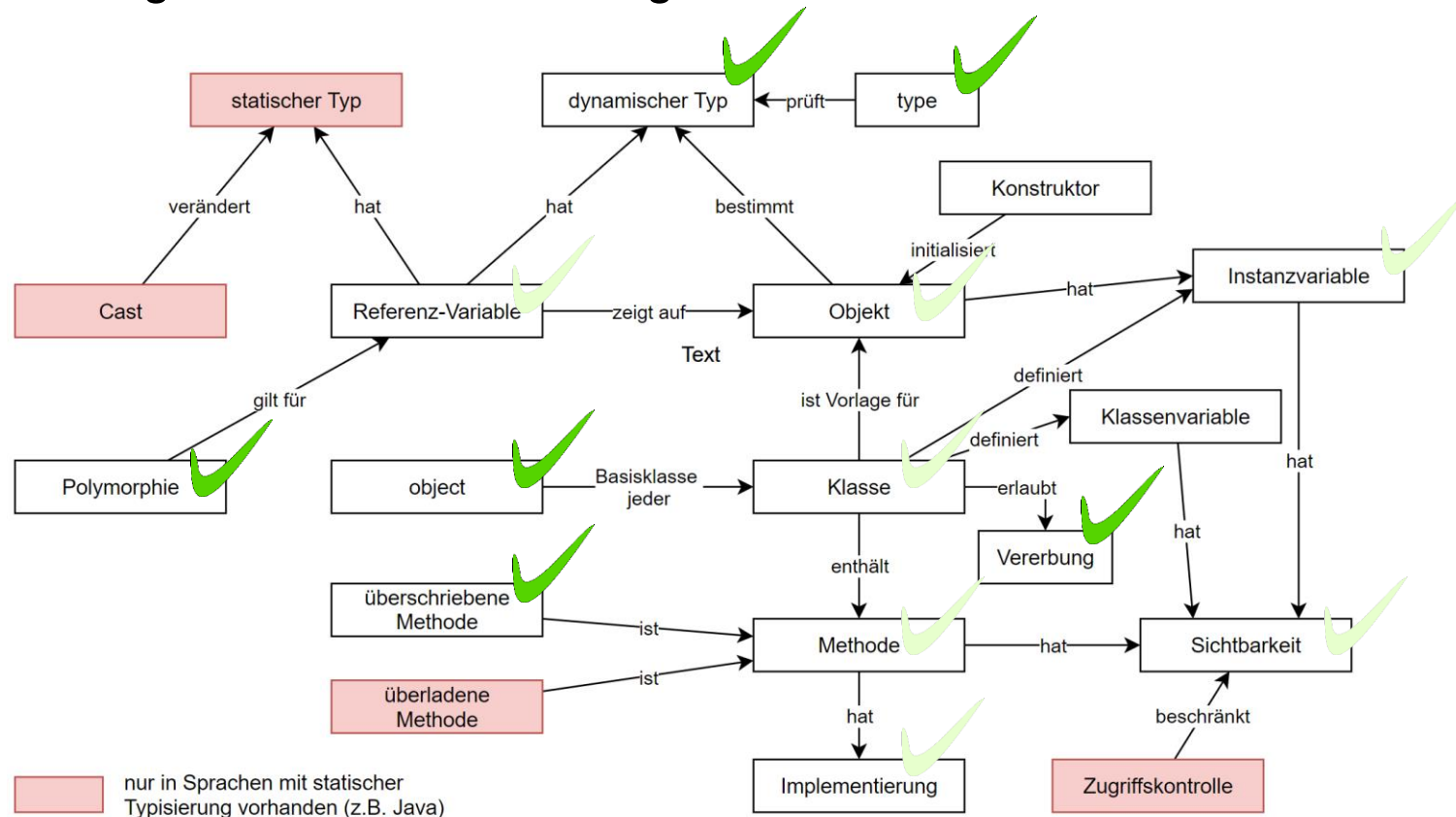
class Dozent2(Person, Angestellter):
    """ Klasse Dozent als Spezialisierung von Person """
    def __init__(self, name=None, geburtsjahr=None, vertragsjahr=None, besoldungsgruppe=None):
        Person.__init__(self, name=name, geburtsjahr=geburtsjahr)
        Angestellter.__init__(self, vertragsjahr=vertragsjahr)
        self._besoldungsgruppe = besoldungsgruppe

    def __str__(self):
        return f"{Person.__str__(self)}, {Angestellter.__str__(self)} " \
            f"und gleichzeitig Dozent mit Besoldungsgruppe {self._besoldungsgruppe}"
    
```

Hinweis zur Identifikation
aufzurufender Methoden
bei Mehrfachvererbung:
[Link](#)

Terminologie – Grundbegriffe

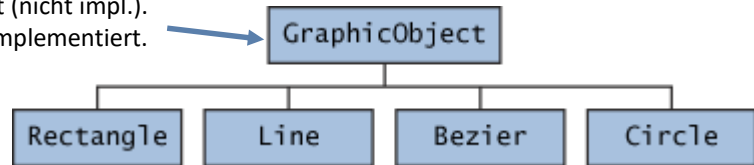
■ Grundlegende Zusammenhänge



Agenda

- ☐ Grundlagen und Terminologie
- ☐ Klassen und Objekte
- ☐ Vererbung und Polymorphie
- ☒ Abstrakte Klassen, Schnittstellen, Exceptions
- ☐ Datenstrukturen und Methoden, Iteratoren, Generatoren
- ☐ Zusammenfassung & Ausblick

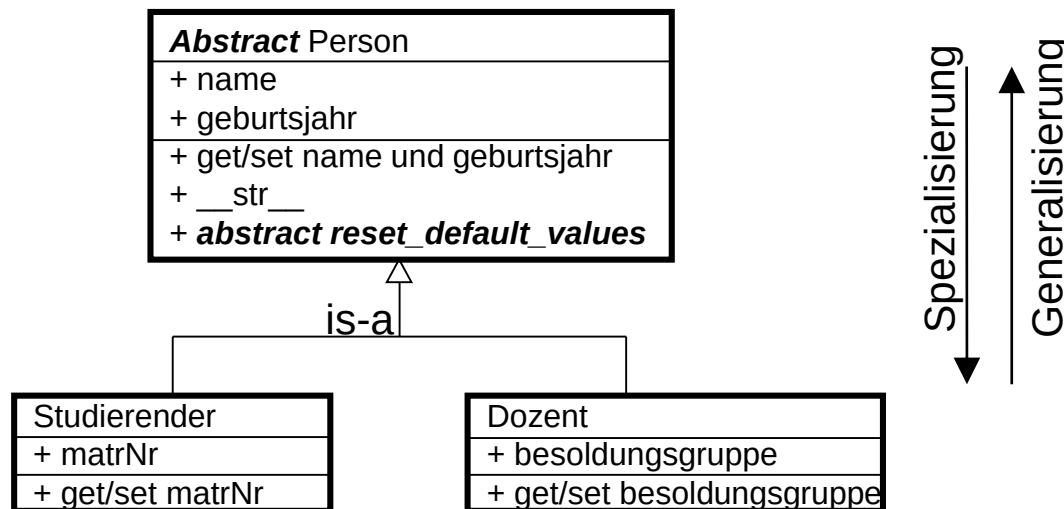
Abstrakte Klassen

- Abstrakte Klassen sind spezielle, nicht instanziiierbare Klassen
 - Es lassen sich also keine Objekte von ihnen erzeugen.
 - Sie dienen somit lediglich als **Strukturelement** innerhalb einer Klassenhierarchie.
 - Definieren Vorschriften, welche Eigenschaften erbende Subklassen haben und welche Funktionalitäten implementiert werden müssen.
- Zweck: Klassen mit gemeinsamen Eigenschaften zusammenfassen
 - Also ähnliche Motivation wie Vererbung, aber die **Implementierung von Methoden existiert auf Superklassen-Ebene nicht zwingend**, sondern wird (teilweise wo gewünscht) den Subklassen überlassen.
 - Wird eine Methode nicht implementiert, so nennt man sie abstrakte Methode.
 - Methoden in abstrakten Klassen dürfen aber auch implementiert sein!
- Beispiele
 - Klasse „Tier“ hat Methode „makeLaut()“, die den Laut des Tiers ausgibt. Welchen Laut macht ein (allgemeines) Tier? → den Subklassen überlassen!
 - Klasse „GraphicObject“ Methode „draw()“ abstrakt definiert (nicht impl.). Methode „moveTo(deltaX, deltaY)“ implementiert. 

```
graph TD; GraphicObject[GraphicObject] --> Rectangle[Rectangle]; GraphicObject --> Line[Line]; GraphicObject --> Bezier[Bezier]; GraphicObject --> Circle[Circle];
```
 - Datenbanktreiber-Hierarchie

Übung

- Kopieren Sie „hochschule.py“ in eine Datei „hochschule_abstrakt.py“ und arbeiten Sie in dieser.
 - Entfernen Sie die Klassen „Dozent2“ und „Angestellter“.
 - Ändern Sie die Klasse Person so, dass sie eine abstrakte Klasse wird.
 - Definieren Sie eine abstrakte Methode `reset_default_values`, die von jeder Subklasse zu implementieren ist. Nutzen Sie dazu den Decorator `@abstractmethod`
 - Es sollen die Attribute auf (von Ihnen selbst definierbare) Standardwerte zurückgesetzt werden.
 - Vor deren Implementierung: Versuchen Sie „Person“ und die Subklassen zu instanziiieren.
 - Implementieren Sie nun die Methoden und versuchen Sie erneut die Instanziierung.
 - Diskutieren Sie einen alternativen Programmieransatz (insb. ohne abstrakte Methode).



Übung

■ Lösungsvorschlag

```
from abc import ABC, abstractmethod
```

Notwendige imports, da Python das Konzept der abstrakten Klassen nicht inkludiert, sondern dieses erweitert wurde.

```
class Person(ABC):
```

Als abstrakt zu definierende Klassen erben von ABC (Abstract Base Class)

```
    """ Abstrakte Klasse zur Repräsentation von Personen.
        Die Methode reset_default() ist von den Subklassen zu implementieren """
```

```
    def __init__(self, name=None, geburtsjahr=None):
        self._name = name
        self._geburtsjahr = geburtsjahr
```

```
    ...
```

Decorator genutzt für abstrakte Methoden. Mit pass die Implementierung übergehen.

```
    @abstractmethod
```

```
    def reset_default_values(self):
        pass # in der Superklasse gibt es keine Implementierung
```

```
class Studierender(Person):
    """ Klasse Studierender als Spezialisierung von Person """
    ...
```

```
    def reset_default_values(self):
        """ Zurücksetzen der Attribute auf Standardwerte """
        self.name = "Standard-Studierender"
        self.geburtsjahr = None
        self.matrNr = None
```

```
class Dozent(Person):
    """ Klasse Dozent als Spezialisierung von Person """
    ...
```

```
    def reset_default_values(self):
        """ Zurücksetzen der Attribute auf Standardwerte """
        self.name = "Standard-Dozent"
        self.geburtsjahr = 1900
        self.besoldungsgruppe = None
```

Die Implementierung in den Subklassen erfolgt so wie „normales“ Überschreiben von geerbten Methoden.

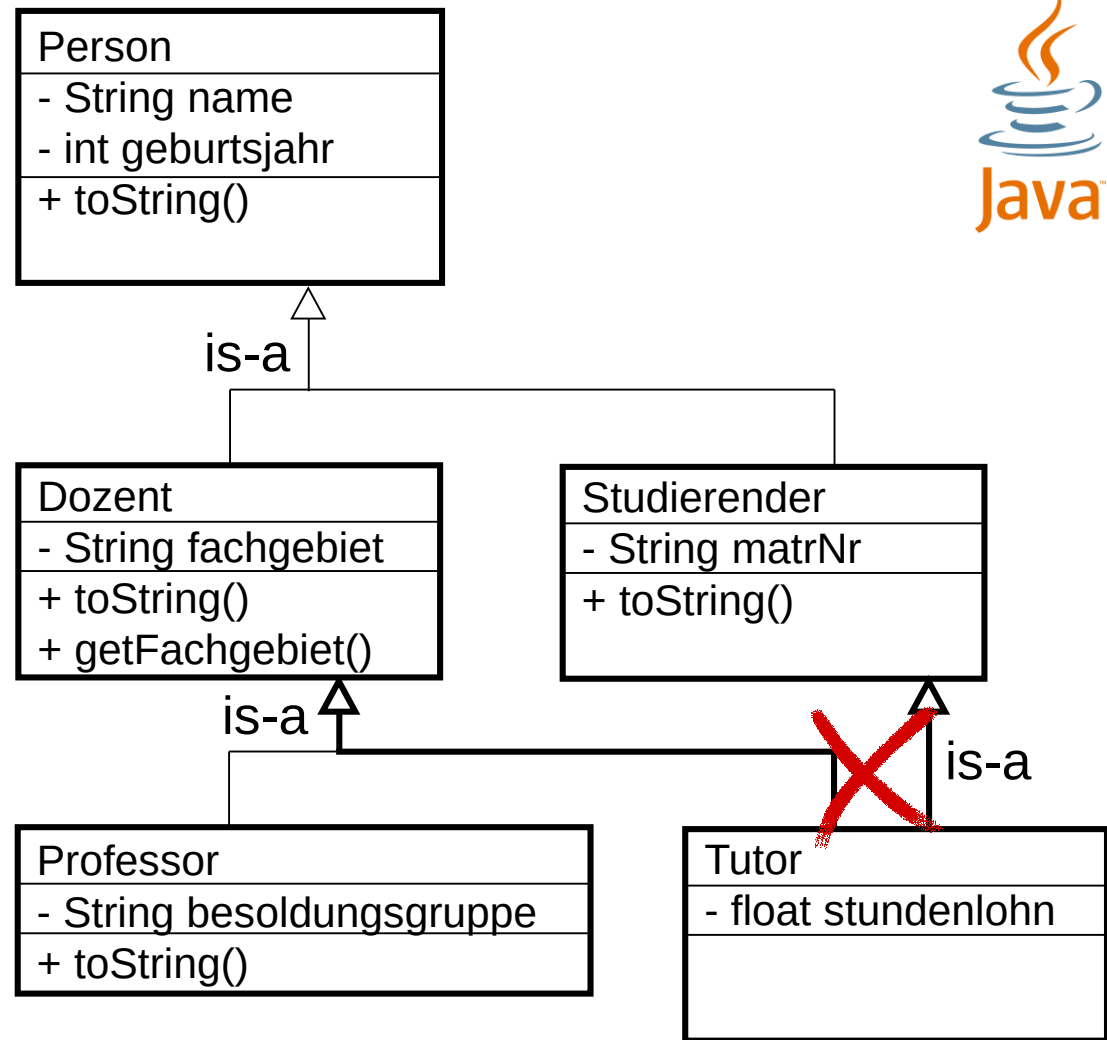
Schnittstellen in der OOP

- Schnittstellen (=Interfaces) sind rein abstrakte Klassen
 - Definition von Methoden, die Klassen anzubieten haben.
 - Eine Klasse „implementiert“ dann ein Interface.
 - Alle Klassen, die ein Interface implementieren, können gleich verwendet werden (Garantie).
- Sie ermöglichen „Programmierung nach Vertrag“
 - Die Schnittstelle gibt vor was zu implementieren ist. Die implementierende Klasse setzt die eigentliche Funktionalität (das „wie“) um.
 - Das „echte“ Design-by-Contract Prinzip bedingt aber mehr als die syntaktische Definition eines Interfaces.
 - Semantik muss klar sein, Vor- und Nachbedingungen müssen definiert sein → meist nur informell in der Spezifikation hinterlegt.
- In Programmiersprachen ohne Möglichkeit zur Mehrfachvererbung können Interfaces verwendet werden, um Kompatibilität zwischen Klassen zu definieren, die nicht voneinander erben.

Problem der Mehrfachvererbung in manchen Sprachen



- Viele Programmiersprachen lassen Mehrfachvererbung nicht zu
 - Beispiel: Java
 - Python erlaubt dies!
- Hier können Interfaces ein Stück helfen.

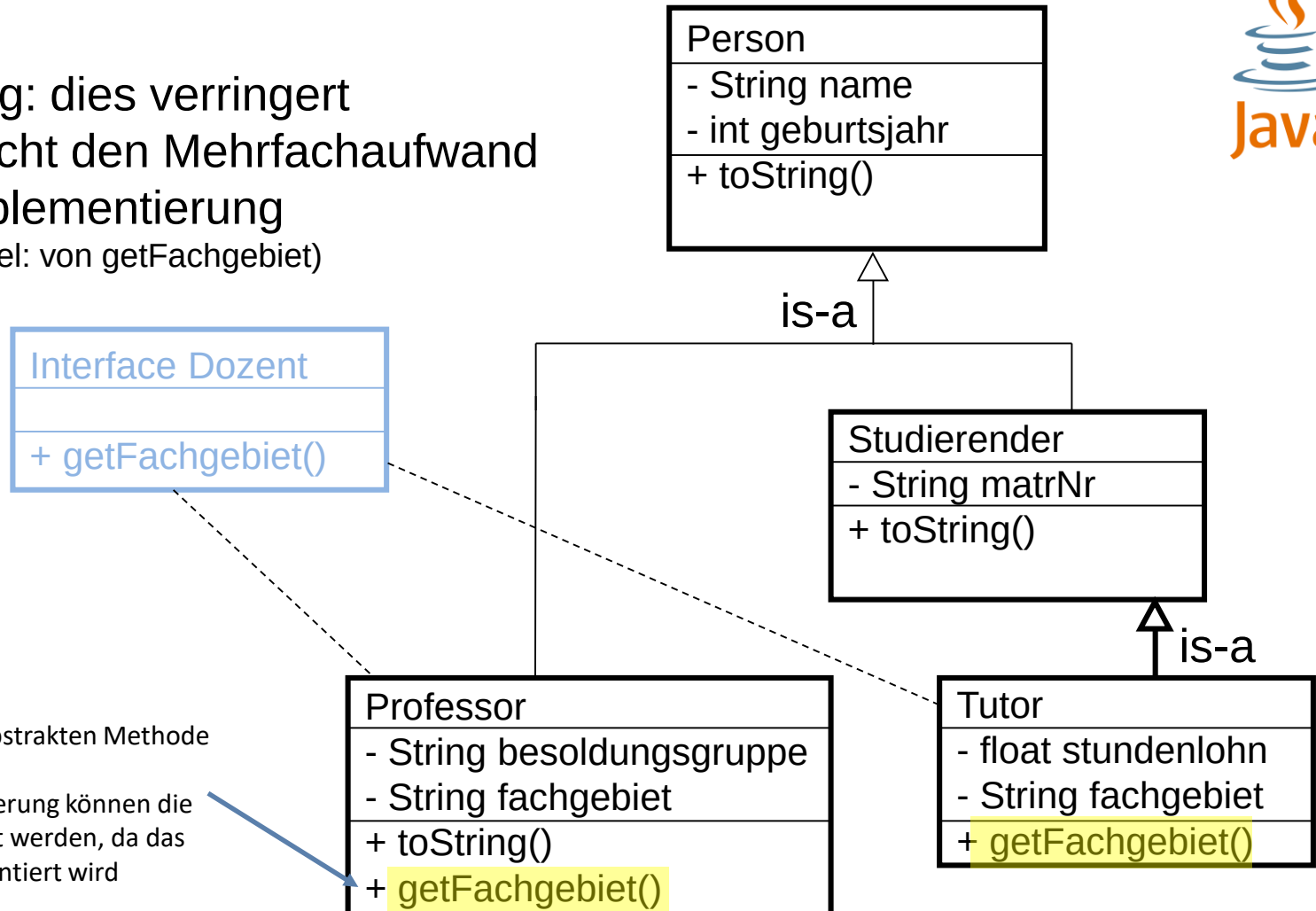


~~`public class Tutor extends Student, Dozent {`~~

Problem der Mehrfachvererbung in manchen Sprachen



- Achtung: dies verringert aber nicht den Mehrfachaufwand der Implementierung (im Beispiel: von `getFachgebiet`)



Implementierung der abstrakten Methode aus Interface „Dozent“.
Ohne diese Implementierung können die Klassen nicht verwendet werden, da das Interface nicht implementiert wird (Kompilierfehler in Java).

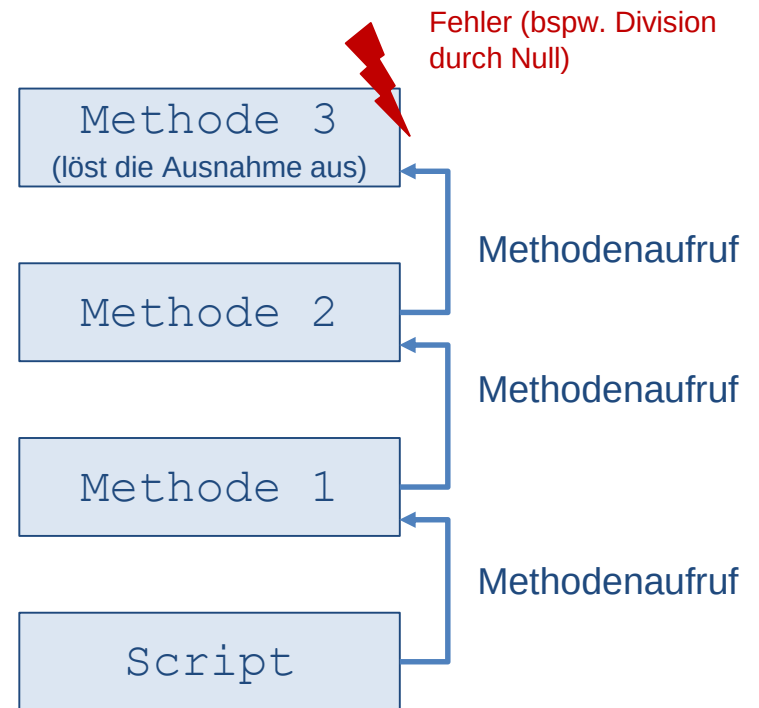
```
public class Tutor extends Student implements Dozent {
```

Schnittstellen in der OOP

- In Python gibt es das Statement „interface“ nicht.
 - So „formell“ wie in bspw. Java (mit statischer Typisierung) können wir also Schnittstellen nicht spezifizieren.
 - Aber: wir könnten die Möglichkeiten von abstrakten Klassen verwenden.
 - Die Notwendigkeit für Interfaces in Python ist aber nicht gegeben
 - Mehrfachvererbung möglich
 - Dynamische Typisierung anstelle von statischer
- Der Code wird so aber „impliziter“, da Schnittstellen nicht klar in der Sprache selbst spezifiziert werden können bzw. müssen (vgl. „[duck typing](#)“).

Exceptions

- Eine Ausnahme (=Exception) ist ein Ereignis, das während der Ausführung eines Programms auftritt und den normalen Ablauf der Befehle in dem Programm unterbricht.
- Konzept und Ideen von Exceptions
 - Trennung des „normalen“ Codes von dem Code, der für die Fehler und Ausnahmebehandlung zuständig ist.
 - Gruppierung verschiedener Fehlerarten und Bildung von „Fehler-Hierarchien“.
 - Das Auftreten eines Fehlers wird nach „oben“ propagiert, d.h. aufrufende Methoden bekommen mit, dass in einer von ihnen aufgerufenen Methode ein Fehler auftrat (inkl. der Fehlerart).
 - In vielen modernen Programmiersprachen werden Exceptions eingesetzt.



Übung

- Legen Sie eine Datei „exceptions.py“ an.
 - Führen Sie fehlerhaften Code aus (bspw. Teilen durch 0, import nicht existierender Packages, Anlegen von Objekten ohne existierende Klasse, Dateizugriff auf eine nicht existierende Datei).
 - Prüfen Sie die jeweiligen Exceptions, die geworfen werden.
 - Betrachten die entsprechende Exception-Dokumentation: <https://docs.python.org/3/library/exceptions.html>

```
# ZeroDivisionError
x = 10
y = 0
z = x / y # y ist 0 --> unzulässig

# ModuleNotFoundError
import notexisting # Package existiert nicht

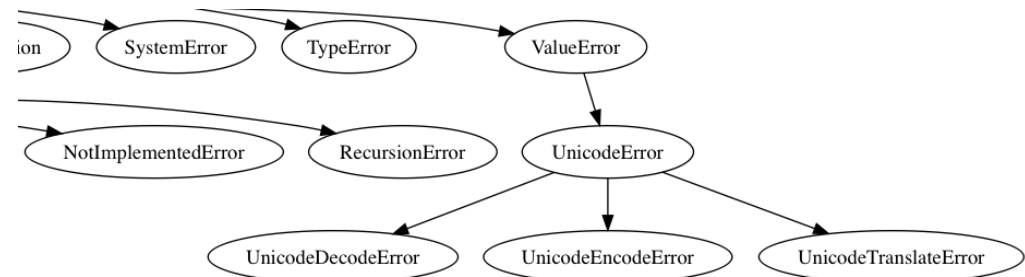
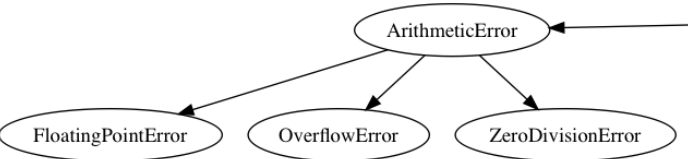
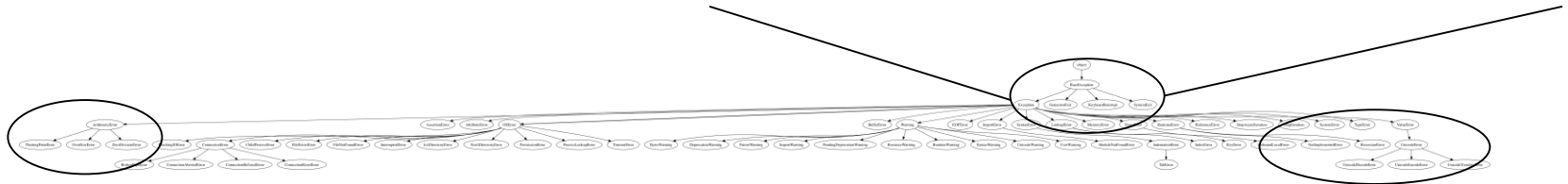
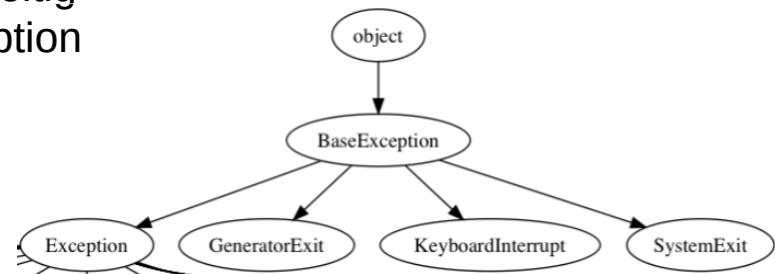
# NameError
x = NotExistingClass() # Anlegen eines Objekts ohne existierende Klasse

# FileNotFoundError
f = open("notexistingfile.txt", "r")
```

Exceptions – Klassenhierarchie

- Alle Exceptions in Python erben von „BaseException“.
 - Ein ZeroDivisionError ist somit gleichzeitig ArithmeticError, Exception, BaseException (und auch object!)

Vgl. <https://docs.python.org/3/library/exceptions.html>



Exceptions – Handling von Ausnahmen

■ Handling von Exceptions

- Exceptions erlauben es, (erwartbare) Fehler von der eigentlichen Programmlogik zu trennen.
- Insbesondere bei Datei-/Netzwerk-/Datenbankzugriffen ist immer mit einem Fehler zu rechnen (z.B. „Datei nicht gefunden“).
- Es muss also ein Fehlerhandling geben, damit das Programm nicht abstürzt (bspw. nur weil die Datei nicht gefunden wurde) → „Fangen (=catch) der Exception“

■ In Python: try: ... except: ...

```
# Einleiten des kritischen Code-Blocks
try:
    a = int(input("Dividend:")) # könnte auch vor dem "try" stehen!
    b = int(input("Divisor:")) # könnte auch vor dem "try" stehen!
    print(f"Quotient = {a/b}") # muss in den try-except-block!
except:
    print("Unzulässige Eingabe") # wird bei Fehler ausgeführt
```

- Wichtig: hier wird eine beliebige Exception „gefangen“.
 - Wir können nicht sicher sein, dass es die unzulässige Eingabe war, die zur Exception führte
→ Mechanismus nötig, um dediziert die Art der zu fangenden Exception anzugeben
→ Exception-Hierarchie!

Exceptions – Spezielle Exceptions verarbeiten

- Handling von spezifischen Exceptions
 - Um das Fehlerhandling genau auf die jeweilige Fehlerart ausrichten zu können, sind mehrere „except“-Blöcke möglich:
- In Python: try: ... except <ExceptionTyp>: ...

```
# Einleiten des kritischen Code-Blocks
try:
    a = int(input("Dividend:")) # könnte auch vor dem "try" stehen!
    b = int(input("Divisor:")) # könnte auch vor dem "try" stehen!
    print(f"Quotient = {a/b}") # muss in den try-except-block!
    print(f"Summe = {a+b}") # muss in den try-except-block!
except ValueError:
    print("Zahlenkonvertierung fehlgeschlagen")
except ZeroDivisionError:
    print("Division durch 0 unzulässig")
except:
    print("Etwas anderes ist nicht in Ordnung.")
```

- Probieren Sie unterschiedliche Fehler aus!

Exceptions – Weitere Möglichkeiten

- „else“ (optional!)
 - Der Code-Block wird ausgeführt, wenn im try-Block alles ohne Exceptions durchlief
- „finally“ (optional!)
 - Der Code-Block wird immer ausgeführt (nach try, else und except, auch wenn Exceptions geworfen wurden)
 - Wird zum „Aufräumen“ verwendet, bspw. Schließen von Dateien.

```
# Einleiten des kritischen Code-Blocks
try:
    a = int(input("Dividend:")) # könnte auch vor dem "try" stehen!
    b = int(input("Divisor:")) # könnte auch vor dem "try" stehen!
    print(f"Quotient = {a/b}") # muss in den try-except-block!
    print(f"Summe = {a+b}") # muss in den try-except-block!
except ValueError:
    print("Zahlenkonvertierung fehlgeschlagen")
except ZeroDivisionError:
    print("Division durch 0 unzulässig")
except:
    print("Etwas anderes ist nicht in Ordnung.")
else:
    print("Keine Fehler")
finally:
    print("Wird immer ausgeführt")
```

Exceptions – Weitere Möglichkeiten

- Zugriff auf das Exception-Objekt
 - Ermöglicht klare Fehlermeldungen:

```
# Einleiten des kritischen Code-Blocks
try:
    a = int(input("Dividend:")) # könnte auch vor dem "try"
    stehen!
    b = int(input("Divisor:")) # könnte auch vor dem "try"
    stehen!
    print(f"Quotient = {a/b}") # muss in den try-except-
    block!
    print(f"Summe = {a+b}") # muss in den try-except-block!
except ValueError as e:
    print(f"Zahlenkonvertierung fehlgeschlagen ({e})")
except ZeroDivisionError as e:
    print(f"Division durch 0 unzulässig ({e})")
except Exception as e:
    print(f"Etwas anderes ist nicht in Ordnung ({e})")
else:
    print("Keine Fehler")
finally:
    print("Wird immer ausgeführt")
```

- Insbesondere interessant für das Erzeugen eigener Exception-Strukturen (s.u.) und das „Werfen“ von Exceptions mit konkreten Fehlermeldungen.

Exceptions – Weitere Möglichkeiten

- Beispiel: Nutzung des Exception Konzepts für den Kontrollfluss
 - Exceptions explizit „Werfen“: Nutzung des raise-Statements
 - Mehrere Exception-Typen prüfen: Nutzung eines Tupels
 - Dies ermöglicht sinnvollen, robusten Code, trotz fehlerhafter Eingaben
 - Aber Achtung: eventuelle Seiteneffekte sollte berücksichtigt werden (da „NaN“ in die Liste eingesetzt wird statt eine Exception zu werfen).

```
def get_ratios(l1, l2):  
    """ Assumption: l1 and l2 are lists of equal length containing numbers  
    Returns: a list containing the ration l1[i]/l2[i] for each list element i """  
    ratios = [] # holds the ratios to be returned  
  
    if len(l1) != len(l2):  
        raise ValueError('lists have different lengths')  
  
    for idx, el1 in enumerate(l1):  
        try:  
            ratios.append(el1 / l2[idx])  
        except (ZeroDivisionError, ValueError, TypeError) as e:  
            # handle issues with types and values by adding nan to the returning list  
            ratios.append(float('nan')) # add nan = not a number  
        except Exception as e:  
            # default error in case of any other issue - raise exception  
            raise ValueError(f'get_ratios called with bad arguments: {e}')  
    return ratios  
  
print(get_ratios([1,'a',3], [0,0,1]))
```

Exceptions – Eigene Fehlerstrukturen

■ Erzeugung eigener Exception-Klassen

- Durch Definition einer Klasse und Erben von einer existierenden Exception können eigene Exceptions definiert werden.
 - Nützlich, um Software-spezifische Exception-Strukturen aufzubauen.
 - „Sprechende“ Exception-Namen: Code lesbarer machen
 - Spezielle Fehlersituationen individuell behandeln
 - Es können eigene Fehlermeldungen definiert werden (durch Anpassung des Konstruktors).
- Beispiel: Das Gehalt muss in einer vordefinierten Range liegen:

```
class SalaryNotInRangeError(ValueError):  
    """ Exception raised for errors in the input salary.  
    Assumptions: raising the exception requires the attribute salary and  
    optionally message for a particular explanation of the error  
    """  
  
    def __init__(self, salary, message="Salary is not in range (5000, 15000)":  
        self.salary = salary  
        self.message = message  
        super().__init__(self.message)  
  
salary = int(input("Enter salary amount: "))  
if not 5000 < salary < 15000:  
    raise SalaryNotInRangeError(salary)
```

- Aufrufende Methoden können die Exception dann verarbeiten (anstelle Programmabsturz)

Exceptions – Weitere Möglichkeiten

- Kontext „testgetriebene Entwicklung“ / „test-driven development, TDD“
 - Bei der agilen Entwicklung von Software eingesetzt.
 - Programmierer erstellt Softwaretests vor den zu testenden Komponenten

- Beispiel: Test einer Klasse für die Multiplikation

- Anforderungen: Objekte sollen per Konstruktor mit zwei Attributen angelegt werden
- Die Methode „multiply“ soll beide Attribute multiplizieren
- Eine mögliche Definition eines Tests sieht so aus:

```
import unittest
class TestMultiplierMethods(unittest.TestCase):

    def test_multiply(self):
        mult1 = Multiplier(10, 20.2)
        mult2 = Multiplier(-2, 5)
        mult3 = Multiplier(-2.2, -5)
        self.assertEqual(mult1.multiply(), 202)
        self.assertEqual(mult2.multiply(), -10)
        self.assertEqual(mult3.multiply(), 11)

unittest.main()
```

- Was könnte man noch testen?

Exceptions – Weitere Möglichkeiten

■ Programmierung der Klasse:

```
class Multiplier():
    def __init__(self, x, y):
        self._x = x
        self._y = y

    def multiply(self):
        return self._x * self._y
```

■ Bei Ausführung von „unittest.main()“ wird die Klasse geprüft.

Ran 1 test in 0.000s

OK

■ Welche Rolle spielen Exceptions dabei?

- Ändern Sie die Implementierung von „multiply“ von Produkt auf Summe.
- Kommentieren Sie unittest.main() aus.
- Führen Sie folgendes aus: `assert(Multiplier(10, 20).multiply() == 200)`
- Die Folge sollte ein „AssertionError“ sein

→ Tests in Python (und anderen Sprachen) basieren auf Exceptions!

→ Das unittest-Framework bietet eine Vielzahl von Tests! vgl. <https://docs.python.org/3/library/unittest.html>

Agenda

- ☐ Grundlagen und Terminologie
- ☐ Klassen und Objekte
- ☐ Vererbung und Polymorphie
- ☐ Abstrakte Klassen, Schnittstellen, Exceptions
- ☒ Datenstrukturen und OOP, Iteratoren
- ☐ Zusammenfassung & Ausblick

Datenstrukturen und OOP

- „In Python everything is an object.“

- Auch „int“, „float“, „str“ etc. sind Klassen.
- Man bemerkt die Erzeugung eines Objekts allerdings nicht immer:

```
x = 2                                "hello".capitalize()
print(type(x))
```

- Die Ihnen bekannten Datenstrukturen sind dementsprechend auch Objekte. Beispiele:

- Listen: implizit angelegt über `[]`, aber Konstruktor-Aufruf auch möglich:

```
l = list() # create an empty list
```

- Dictionary: implizit angelegt über `{ }`, aber Konstruktor-Aufruf auch möglich:

```
d = dict() # create an empty dictionary
```

- Tupel: implizit angelegt über `()`, aber Konstruktor-Aufruf auch möglich:

```
t = tuple() # create an empty tuple
```

- Somit definiert die API der jeweiligen Klasse die verwendbaren Methoden auf den Objekten. → wichtig mit der API umzugehen

■ Beispiel: Listen

- Vgl. <https://docs.python.org/3/tutorial/datastructures.html> und <https://docs.python.org/3/library/stdtypes.html#list>

```
# Exemplary list operations
fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi',
          'apple', 'banana']
print(fruits.count('apple'))
print(fruits.count('tangerine'))
print(fruits.index('banana'))
# find next banana starting a position 4
print(fruits.index('banana', 4))
fruits.reverse()
print(fruits)
```

```
fruits.append('grape')
print(fruits)
```

```
fruits.sort()
print(fruits)
```

```
print(fruits.pop())
print(fruits)
```

5.1. More on Lists

The list data type has some more methods. Here are all of the methods of list objects:

list.append(x)

Add an item to the end of the list. Equivalent to `a[len(a):] = [x]`.

list.extend(iterable)

Extend the list by appending all the items from the iterable. Equivalent to `a[len(a):] = iterable`.

list.insert(i, x)

Insert an item at a given position. The first argument is the index of the element before which to insert, so `a.insert(0, x)` inserts at the front of the list, and `a.insert(len(a), x)` is equivalent to `a.append(x)`.

list.remove(x)

Remove the first item from the list whose value is equal to x. It raises a `ValueError` if there is no such item.

list.pop([i])

Remove the item at the given position in the list, and return it. If no index is specified, `a.pop()` removes and returns the last item in the list. (The square brackets around the *i* in the method signature denote that the parameter is optional, not that you should type square brackets at that position. You will see this notation frequently in the Python Library Reference.)

list.clear()

Remove all items from the list. Equivalent to `del a[:]`.

list.index(x[, start[, end]])

Return zero-based index in the list of the first item whose value is equal to x. Raises a `ValueError` if there is no such item.

The optional arguments *start* and *end* are interpreted as in the slice notation and are used to limit the search to a particular subsequence of the list. The returned index is computed relative to the beginning of the full sequence rather than the *start* argument.

list.count(x)

Return the number of times x appears in the list.

list.sort(*, key=None, reverse=False)

Sort the items of the list in place (the arguments can be used for sort customization, see `sorted()` for their explanation).

list.reverse()

Reverse the elements of the list in place.

list.copy()

Return a shallow copy of the list. Equivalent to `a[:]`.

■ Beispiel: Dictionaries

- Vgl. <https://docs.python.org/3/library/stdtypes.html#dict>

```
# dictionary iteration
capitols = {'Germany': 'Berlin', 'France': 'Paris'}
for k, v in capitols.items():
    print(k, v)

# access only the keys or values
print(capitols.keys())
print(capitols.values())

# update / add new dictionary to existing one
capitols.update({'Germany': 'Berlin', 'Ireland': 'Dublin'})
print(capitols)

# "in"-notation to check existence of keys
print("Ireland" in capitols.keys())

# pop-method in analogy to its usage with lists
print(capitols.pop('France'))
print(capitols)
```

...

get(key[, default])

Return the value for *key* if *key* is in the dictionary, else *default*. If *default* is not given, it defaults to `None`, so that this method never raises a `KeyError`.

items()

Return a new view of the dictionary's items ((*key*, *value*) pairs). See the [documentation of view objects](#).

keys()

Return a new view of the dictionary's keys. See the [documentation of view objects](#).

pop(key[, default])

If *key* is in the dictionary, remove it and return its value, else return *default*. If *default* is not given and *key* is not in the dictionary, a `KeyError` is raised.

popitem()

Remove and return a (*key*, *value*) pair from the dictionary. Pairs are returned in [LIFO](#) order.

`popitem()` is useful to destructively iterate over a dictionary, as often used in set algorithms. If the dictionary is empty, calling `popitem()` raises a `KeyError`.

Changed in version 3.7: LIFO order is now guaranteed. In prior versions, `popitem()` would return an arbitrary key/value pair.

...

Datenstrukturen und OOP

- Die vorgestellten Python „Built-in“-Datenstrukturen sind hinsichtlich Handling und Effizienz sehr begrenzt.
 - Im Data Science- / Machine Learning-Kontext werden üblicherweise spezifische Klassen verwendet, die
 - die Daten „intern“ effizienter speichern und für Abfragen bereitstellen können, (→ bspw. NumPy)
 - umfangreiche Analytics-Funktionen und eine saubere Integration in Bezug auf Machine Learning-Methoden bieten (→ bspw. Pandas, das auf NumPy aufbaut)
- Wann immer möglich sollte für Analytics auf die Standard-Datentypen verzichtet und eine Analytics-Klasse verwendet werden
- Insbesondere „for-Loops“ sollten zur Datenmanipulation vermieden werden
 - Dazu später mehr.

Datenstrukturen und OOP

■ Beispiel: „Pandas DataFrame“

- Installieren Sie (falls nicht vorhanden) das Package „seaborn“ in Ihrer Umgebung.

■ Wir können eine tabellarische Datenstruktur in einem pandas DataFrame speichern.

- Dazu wird ein entsprechendes Objekt erzeugt, wenn wir den Konstruktor richtig nutzen:

```
# create example tabular data structure (pandas DataFrame)
import pandas as pd
df_sales = pd.DataFrame([[ 'A', 10], [ 'B', 20], [ 'B', 15], [ 'A', 5]], columns=[ 'product', 'sales'])
print(df_sales)
```

- Wenn wir uns mit der API von pandas DataFrames auseinandersetzen, können wir bspw. Analysen in Analogie zu „SQL Group By“ durchführen:

```
# use exemplary analysis-method on the DataFrame object
print(df_sales.groupby('product').agg({'sales' : ['mean', 'min', 'max']}))
```

- Ein weiteres Beispiel:

```
# load dataset 'mpg' into dataframe
import seaborn as sns
df_mpg = sns.load_dataset('mpg')
print(df_mpg.groupby([ 'origin']).agg({'mpg': 'mean', 'horsepower': 'mean'}))
```

Iteratoren

- Wir haben bereits Beispiele des „Iterierens“ durch Datenstrukturen kennengelernt:

```
fruits = ['orange', 'apple']
for fruit in fruits:
    print(fruit)
```

```
capitols = {'Germany': 'Berlin', 'France': 'Paris'}
for k, v in capitols.items():
    print(k, v)
```

- Die Objekte, die wir mittels for-Schleife auf diese Weise „durchgehen“ nennen sich iterierbar („**iterable**“).
 - Man kann einen Iterator von einem iterierbaren Objekt erzeugen.
 - Dies passiert implizit bei der Verwendung einer for-Schleife.
 - Wir können es auch explizit umsetzen, aber for-Schleifen sind eleganter:

```
# explizite Erstellung und Verwendung
# eines Iterator-Objekts
x = iter(fruits)
print(x)
print(next(x))
print(next(x))
print(next(x)) # kein Objekt mehr! --> Exception
```

```
# so könnte die vollständige
# Iteration umgesetzt werden,
# aber for-loops sind "eleganter"
x = iter(fruits)
try:
    while True:
        print(next(x))
except StopIteration:
    pass
```

Iteratoren

■ Iteratoren und OOP

- Implementieren Klassen beide spezielle Methoden `__iter__` und `__next__`, dann können Objekte der Klassen einfach in for-Schleifen verwendet werden.

```
class Kartenspiel():
    """ Klasse, die ein Standard-Kartenspiel enthält """
    def __init__(self):
        """ Konstruktor: legt ein Standard-Kartenspiel an """
        farben = ['kreuz', 'pik', 'karo', 'herz']
        werte = [7, 8, 9, 10, 'bube', 'dame', 'koenig', 'as']

        self._spielkarten = [] # Liste mit allen Spielkarten im Spiel
        for f in farben:
            for w in werte:
                self._spielkarten.append(Spielkarte2(farbe=f, wert=w))

    def __iter__(self):
        """ Initialisiert den Iterator """
        self._i = 0 # Initialisierung des Iterators (Startindex 0)
        return self

    def __next__(self):
        """ Liefert die nächste Karte bis zum Ende der Liste """
        if self._i < len(self._spielkarten):
            karte = self._spielkarten[self._i]
            self._i += 1
            return karte
        else:
            raise StopIteration()
```

Nutzung:

```
spiel = Kartenspiel()
for karte in spiel:
    karte.drucken()
```

Iteratoren

■ Iteratoren und Datenstrukturen

- Viele Datenstrukturen, die mehrere Elemente enthalten („Container“) implementieren das Iterator-Konzept
- Auch über DataFrames könnte so iteriert werden:

```
# NICHT EMPFOHLEN! Nutzen Sie die pandas-API!  
for row in df_sales.values:  
    print(row)
```

- Für Analyseaufgaben ist diese Art der Iteration und des „Zurückfallens“ von Pandas auf „normale“ Schleifen aber nicht empfohlen.
 - Nutzen Sie stattdessen die pandas-API so weit wie möglich. Diese ist effizienter implementiert!

Agenda

- ☐ Grundlagen und Terminologie
- ☐ Klassen und Objekte
- ☐ Vererbung und Polymorphie
- ☐ Abstrakte Klassen, Schnittstellen, Exceptions
- ☐ Datenstrukturen und OOP, Iteratoren
- ☒ Zusammenfassung & Ausblick



- Sie können nun die grundlegenden Ideen der Objektorientierten Programmierung wiedergeben und kennen die Terminologie
 - Klassen, Objekte, Methoden, Attribute, Vererbung, Polymorphie, ...
- Sie können Klassen implementieren und Objekte anlegen.
 - Sie wissen, wie der Attributzugriff umzusetzen ist, können Konstruktoren definieren und kennen weitere spezielle Methoden wie `__str__` oder `__eq__`. In den Kontext können Sie auch `named` von `positional arguments` unterscheiden.
- Sie wissen, wozu Vererbung und Polymorphie in der OOP genutzt wird und können Vererbungsstrukturen in Python umsetzen.
- Sie kennen weitere Ansätze der OOP, insbesondere abstrakte Klassen und Schnittstellen. Diese können Sie implementieren.
 - Weiterhin ist Ihnen das Exception-Konzept bekannt und Sie sind in der Lage, Exceptions zu verarbeiten sowie eigene Exceptions zu definieren.
- Sie kennen die Zusammenhänge zwischen Datenstrukturen und der OOP und haben erste Methoden aus Analytics-APIs verwendet. Auch Iteratoren können Sie für Ihre Klassen nun umsetzen.

Vielen Dank für Ihre Aufmerksamkeit!

